

ICT1011

Computer Organisation and Architecture

Chapter 2

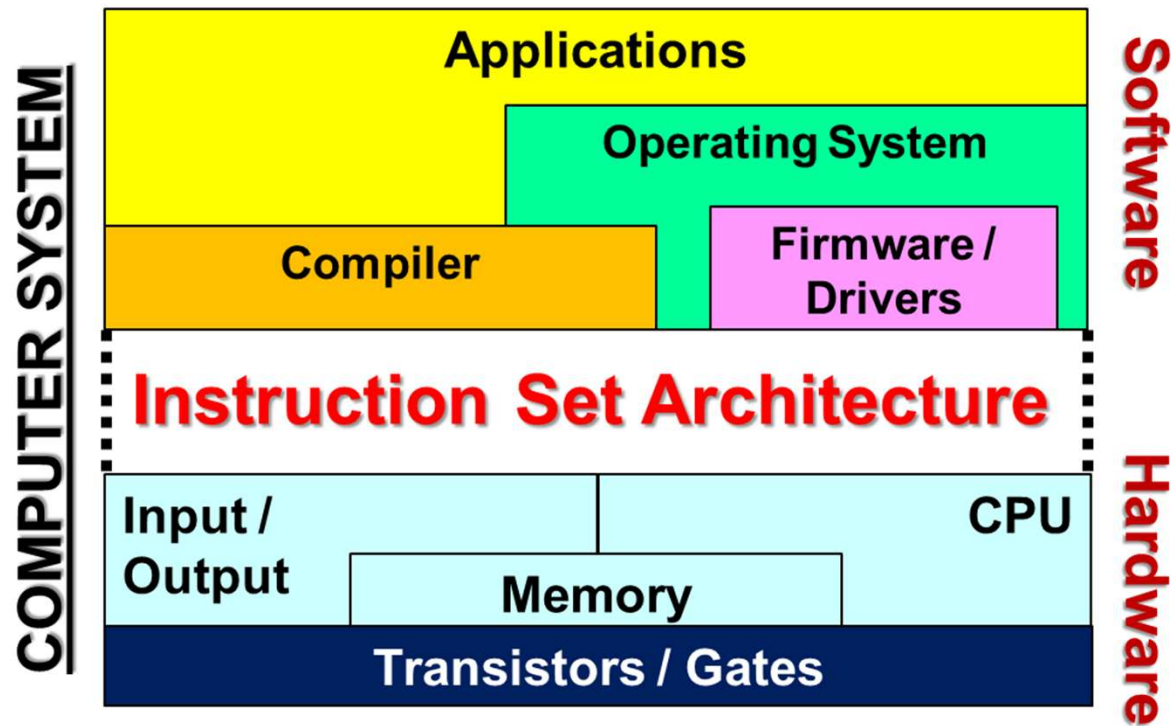
The Central Processing Unit (CPU)

Chapter Contents

- 1. Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
- 2. Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
- 4. Program execution**
 - A typical instruction execution cycle
- 5. Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Chapter Contents

6. Program Development Process



Chapter Contents



<https://ict1011.stevenwong.org/c2t1/>

1. Approaches to Computing

- Programming by hardware versus by software
- von Neumann Architecture
- The Instruction Set Architecture (ISA)

2. Basic components of a microcomputer

- Memory
- Input/Output Interfaces
- CPU

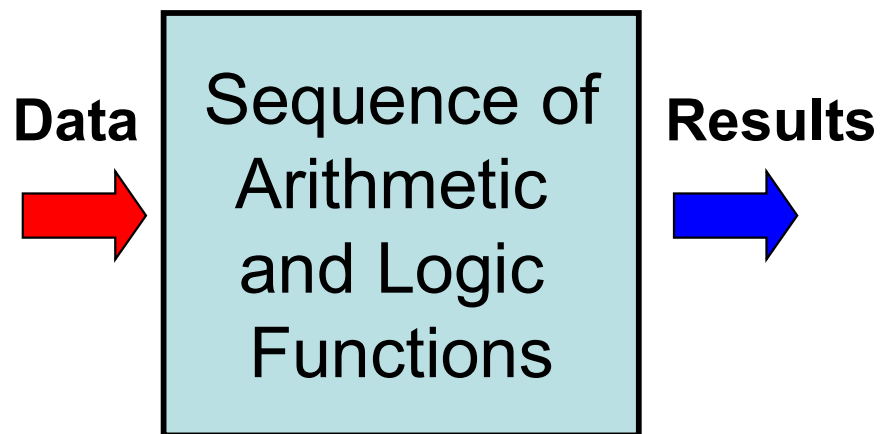
4. Program execution

- A typical instruction execution cycle

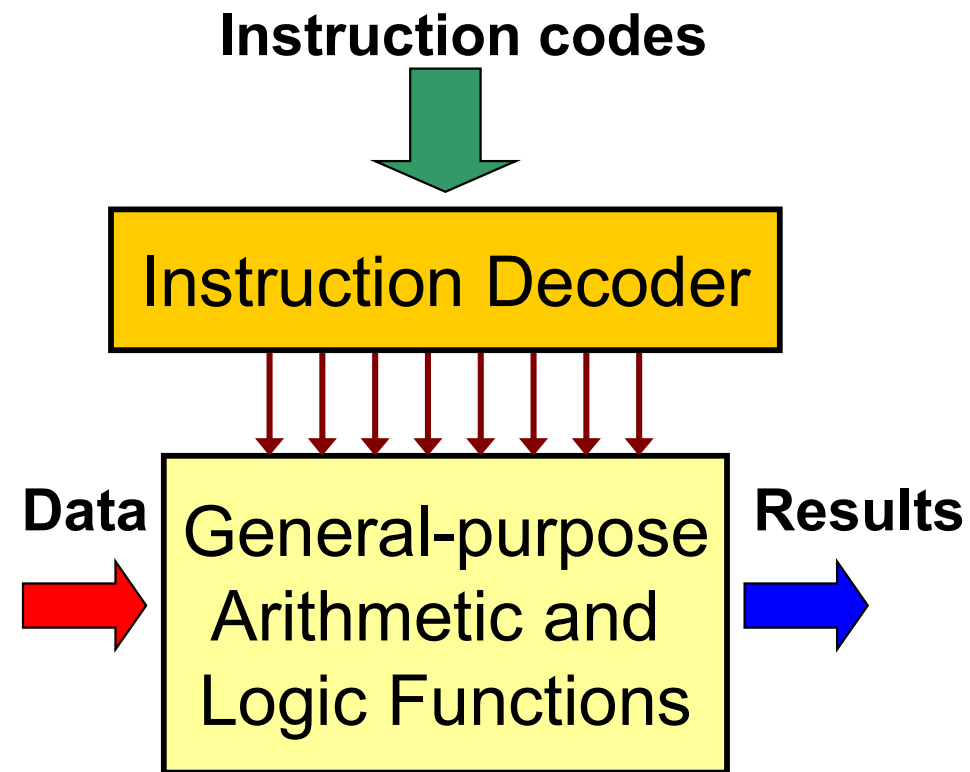
5. Different Computer Architectures

- Harvard (vs von Neumann)
- Flynn's Taxonomy
- RISC and CISC

Limitations of Hardware “Programming”



Programming in Hardware

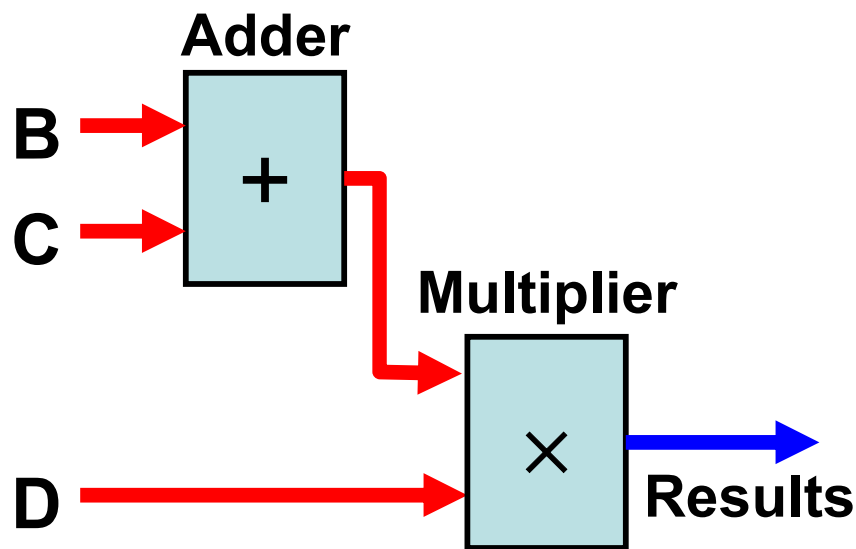


Programming in Software

Limitations of Hardware “Programming”

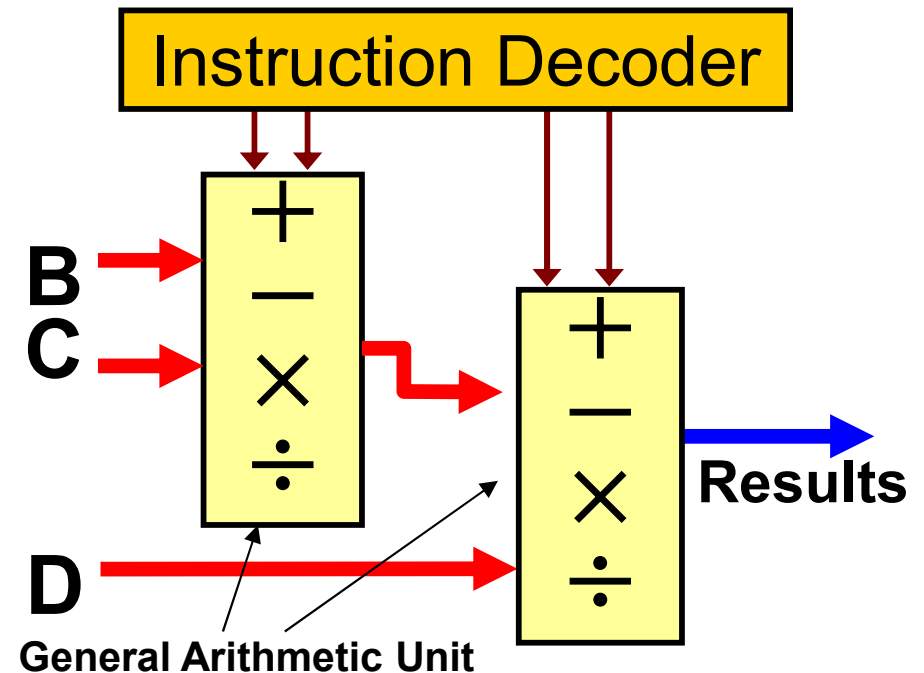
Implementing $A = (B+C) \times D$

Fast computation but very inflexible



Programming in Hardware

Slower and more complex but easily programmable



Programming in Software

Chapter Contents



<https://ict1011.stevenwong.org/c2t2/>

1. Approaches to Computing

- Programming by hardware versus by software
- von Neumann Architecture
- The Instruction Set Architecture (ISA)

2. Basic components of a microcomputer

- Memory
- Input/Output Interfaces
- CPU

4. Program execution

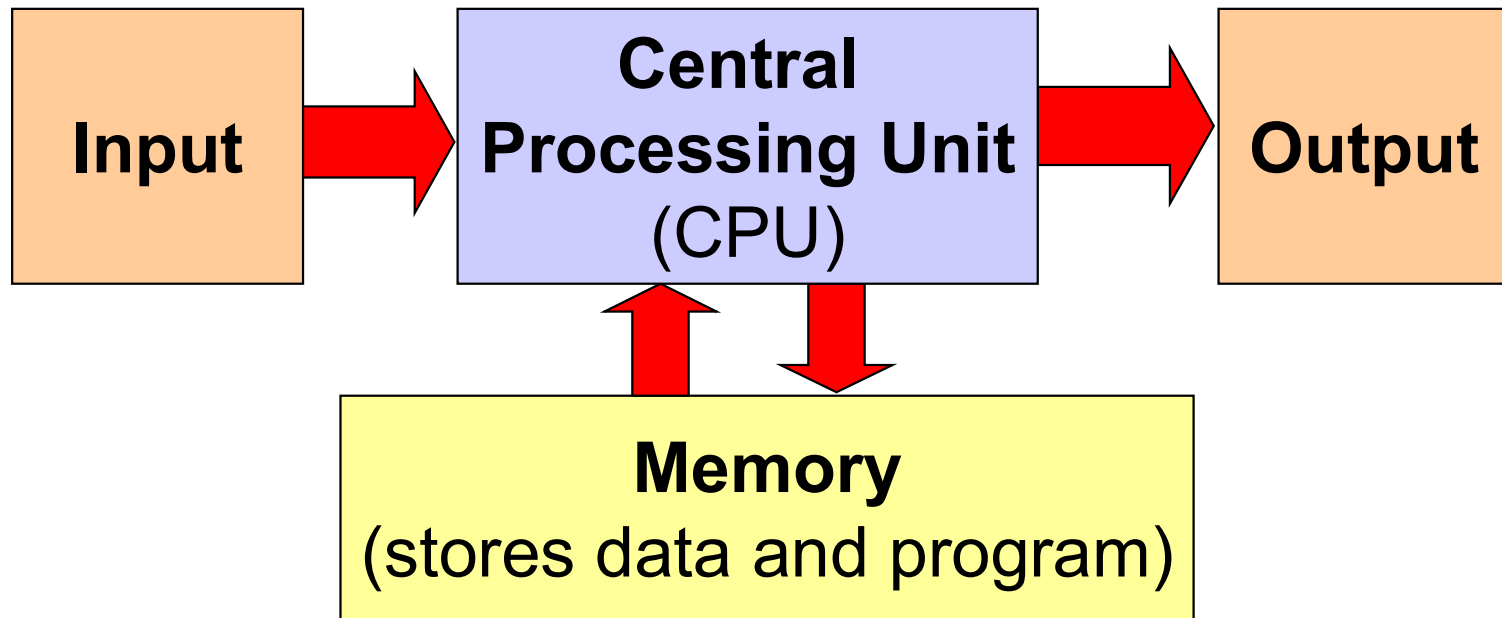
- A typical instruction execution cycle

5. Different Computer Architectures

- Harvard (vs von Neumann)
- Flynn's Taxonomy
- RISC and CISC

Approaches to Computing

The von Neumann Architecture



Most modern day computer design are based on von Neumann's stored program concept:

1. Both data and program are stored in the same memory
2. Contents of memory are addressable by location, without regard to data type
3. Execution occurs sequentially (unless explicitly modified)

Approaches to Computing

Program, Data and Memory

What is a program and what is data?

A **program** is a sequence of instructions.

Data are values these instructions operate on.

What is the memory?

Its a sequential list of addressable storage elements for storing both program and data.

e.g. $B+C$

Address	Content	
0000	+	Instruction
0001	B	Data
0002	C	Data
0003		
0004		
:	:	
:	:	

Memory

Chapter Contents



<https://ict1011.stevenwong.org/c2t3/>

1. Approaches to Computing

- Programming by hardware versus by software
- von Neumann Architecture
- The Instruction Set Architecture (ISA)

2. Basic components of a microcomputer

- Memory
- Input/Output Interfaces
- CPU

4. Program execution

- A typical instruction execution cycle

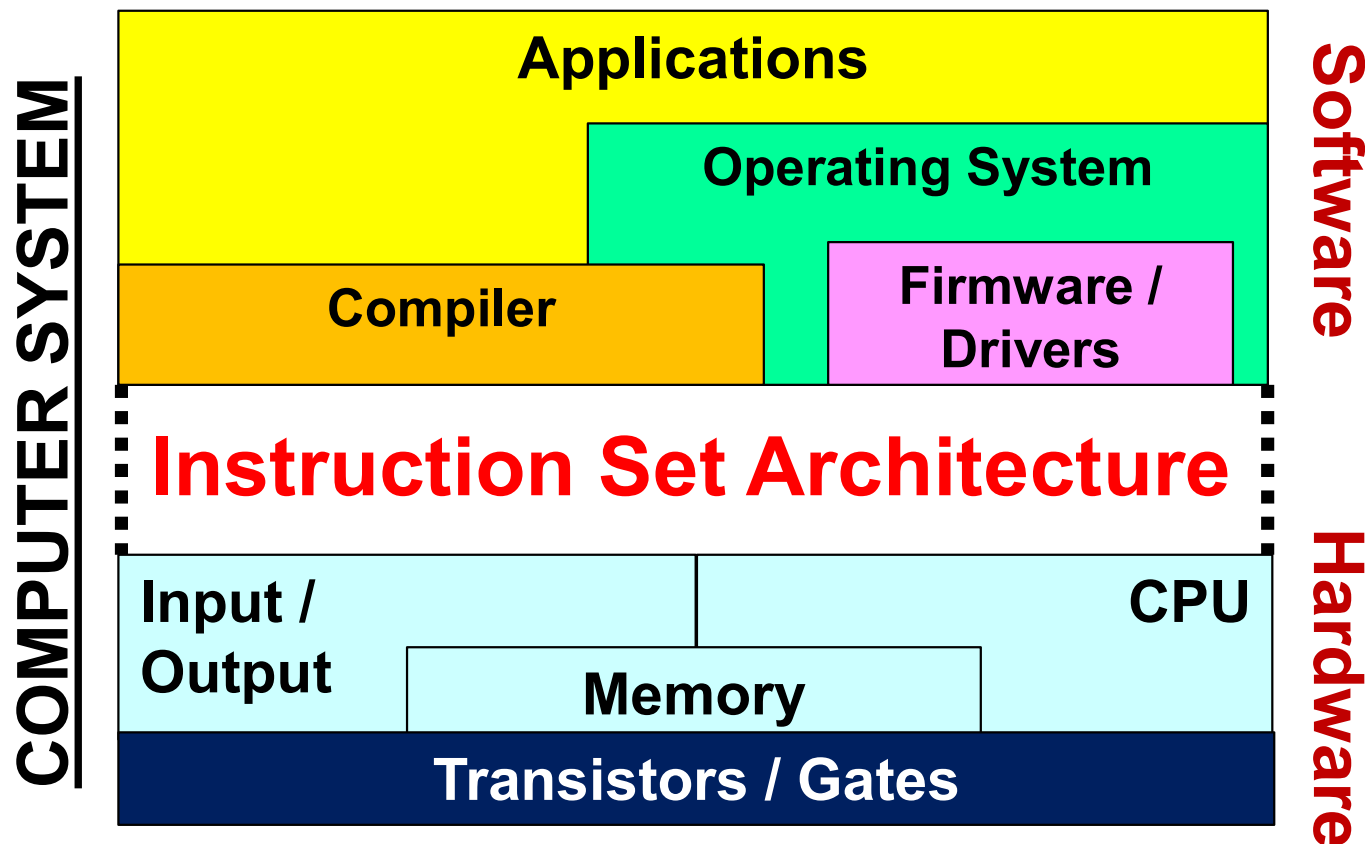
5. Different Computer Architectures

- Harvard (vs von Neumann)
- Flynn's Taxonomy
- RISC and CISC

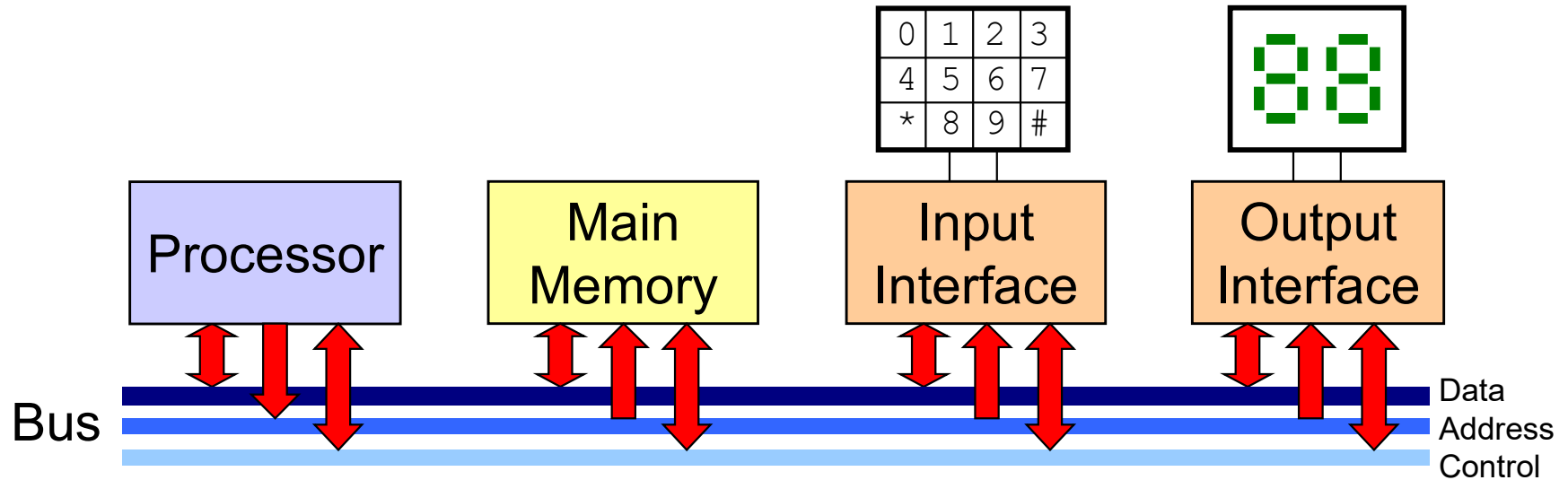
Instruction Set Architecture (ISA)

Instruction Set Architecture (ISA)

The ISA specifies the interface between the Hardware and Software. This includes the low-level machine instructions and data organisation.



Components of a microcomputer



Consist of three main components: **processor**, main **memory** and **I/O** interfaces.

They are interconnected by a **bus** structure, which consist of a collection of wires through which binary information can transferred in parallel.

Chapter Contents



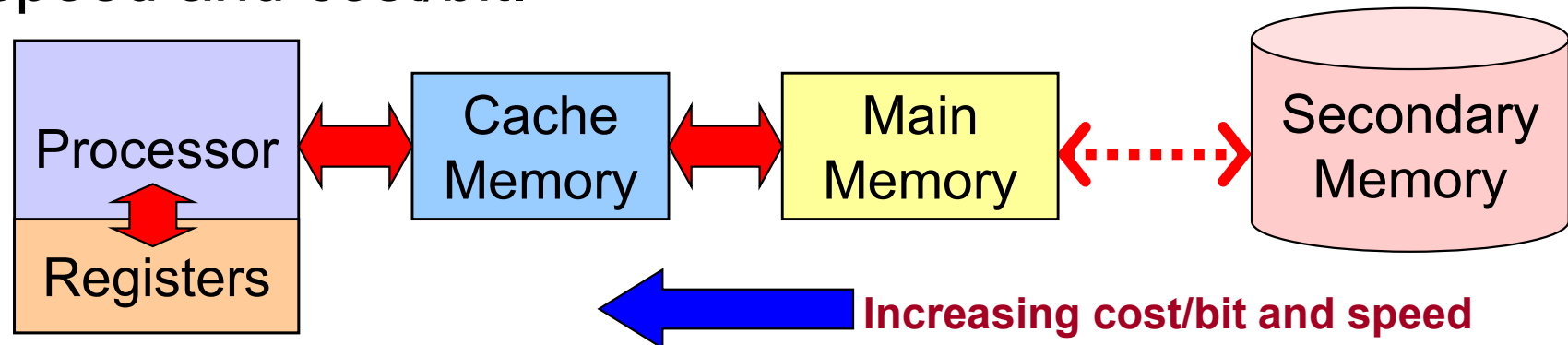
<https://ict1011.stevenwong.org/c2t4/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Memory

Memory Hierarchy

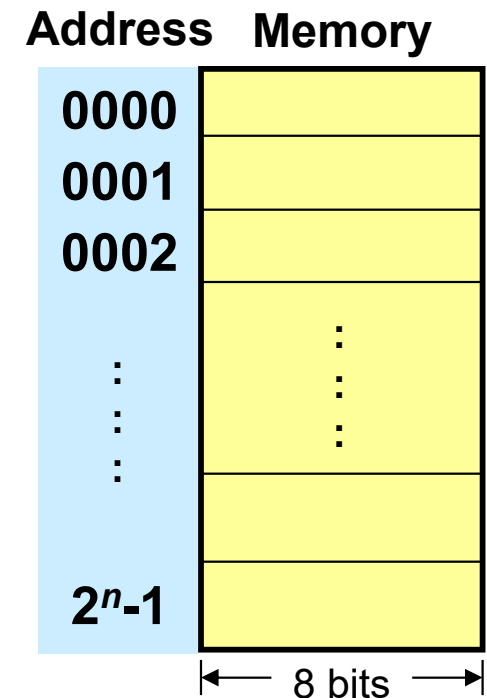
- Memories are generally organized in levels of decreasing speed and cost/bit.



- Registers** Very fast access but limited numbers within CPU. Operates at CPU clock rate (size: 2-128 registers)
- Cache Memory** Fast access static RAM close to CPU. Typical access time 8-35nS (size: 1kB - 512 kB)
- Main memory** Usually dynamic RAM or ROM (for program storage). Typical access time 20-100nS. (size: 1kB – 1GB)
- Secondary Memory** Not always random access but non-volatile. Maybe be based on magnetic or flash technology. Typical access time 5-20mS. (size: 1MB - 80GB)

Characteristics of the Main Memory

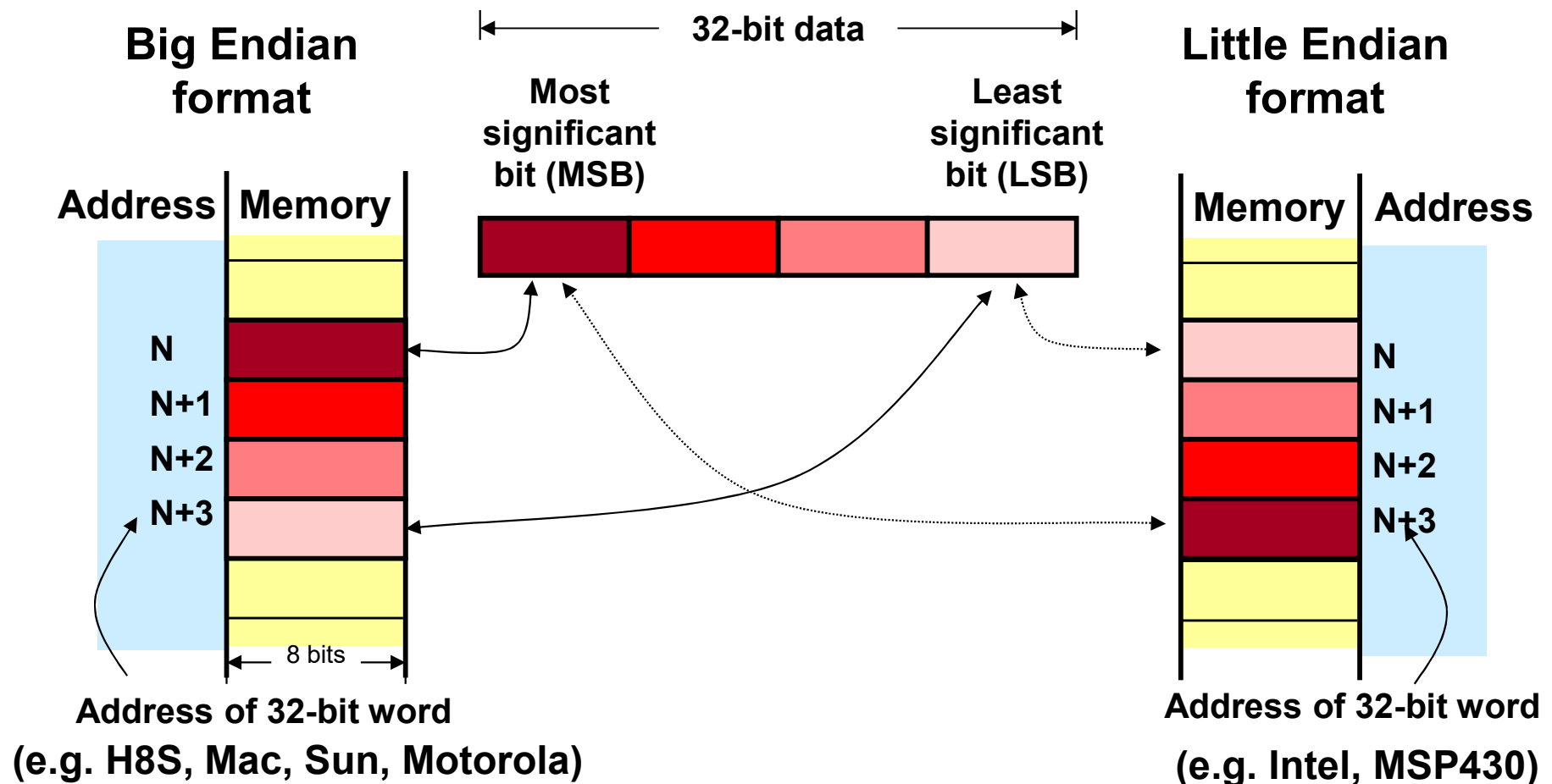
- **Fix-sized** (8-bit) storage location accessible at high speed and in **any order** (i.e. RAM)
- Each byte-sized location has a unique **address** that is accessed by specifying its binary pattern on the **address bus**.
- Main memory size is dependent on number of lines (n) in the address bus. (Memory size = 2^n bytes).
- Memory stores both **data** and **instructions**. If their size > 8 bits, **consecutive** locations are used.



Data Organization in Memory

How is a 32-bit data stored in memory?

There are two ways, depending on the **byte-ordering** of the data in memory



Instruction Organization in Memory

More details from Chpt 3 onwards.

- Instructions are binary encoded and stored in memory. Unlike data, instruction bytes tell the CPU what action to take (i.e. they are **executable**).

- Most instruction formats consist of **two** parts.

Instruction
format

Op-code

Operands

- First part of instruction (**op-code**) specifies the operation to be carried out (e.g. add, subtract).

- Remaining bits (**operands**) specifies the data itself or the location of the data that is to be used in the operation.

Memory

Memory Map

- A memory map is a visual means to show the contents of some consecutive address space.
- Memory maps are usually drawn in byte or word-sized.
- Both address and data are usually represented in **hexadecimal form**.

Address	Content
H' 000000	H' AB
H' 000001	H' CD
H' 000002	H' 12
H' 000003	H' 34
:	:

Byte-sized memory map

Address	Content
H' 000000	H' ABCD
H' 000002	H' 1234
H' 000004	H' 0000
H' 000006	H' 67B5
:	:

Word-sized memory map

Chapter Contents



<https://ict1011.stevenwong.org/c2t5/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Peripherals and I/O Interfaces

Peripherals connects to a computer system in 2 ways:

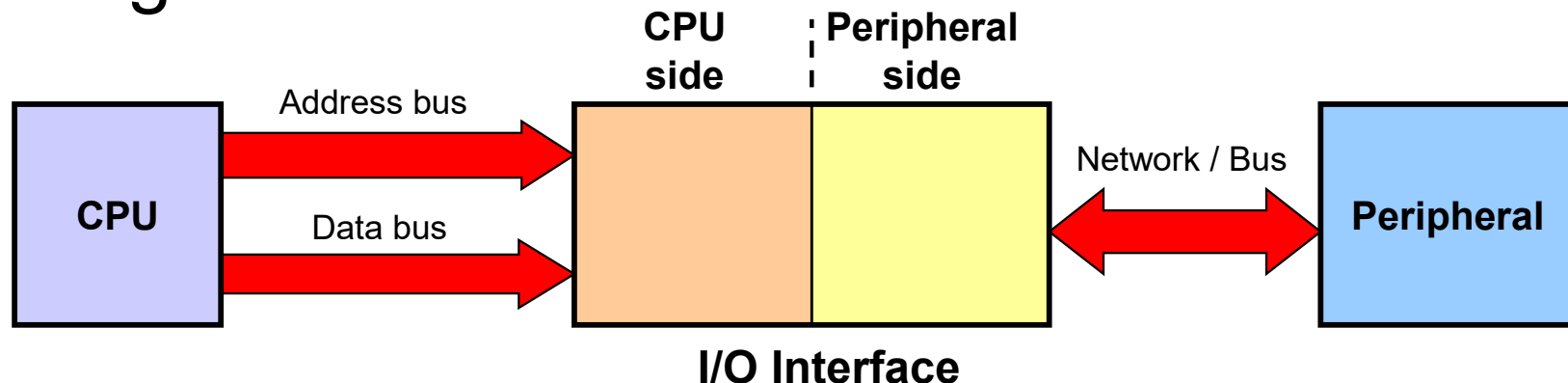
Loosely coupled

- via an **external bus** – USB, SCSI, I²C, Firewire
- via a **network** – Ethernet, ATM, airport (wireless)
- via a **port** – serial (RS232), parallel, PS2

Tightly coupled

- via fast internal bus – graphics & hard-disk controllers

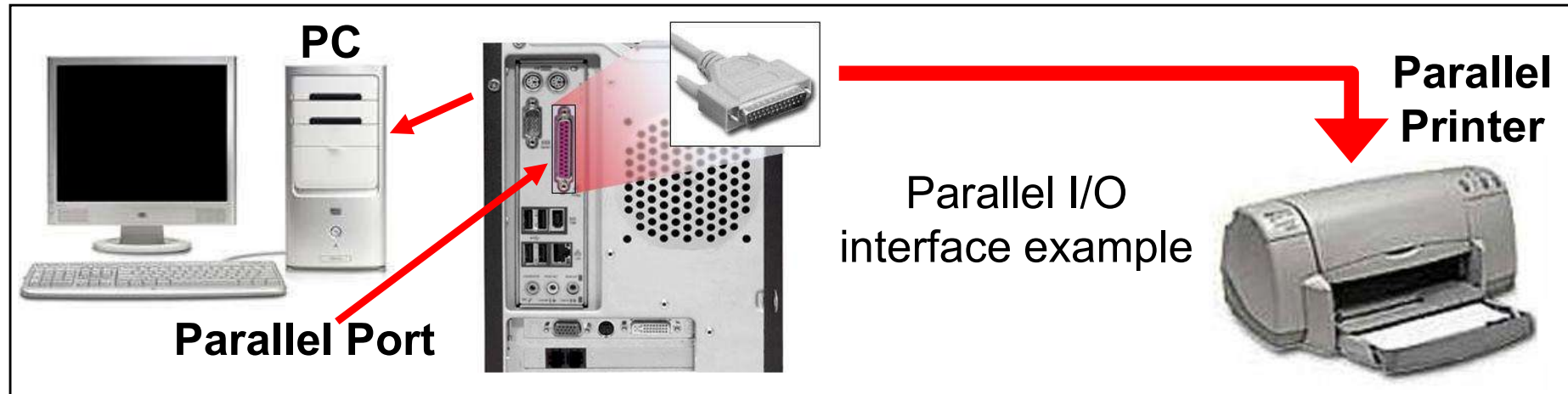
Peripherals generally communicates with the CPU through **I/O interfaces**.



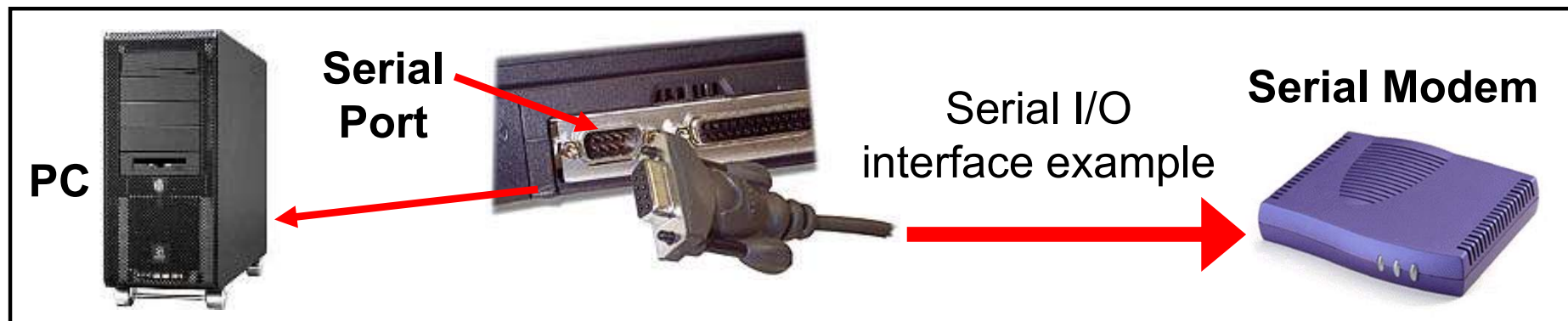
Input / Output I/O Interfaces

The two basic I/O interfaces covered are:

Parallel Input and Output Interface



Serial Communication Interface



Chapter Contents



<https://ict1011.stevenwong.org/c2t6/>

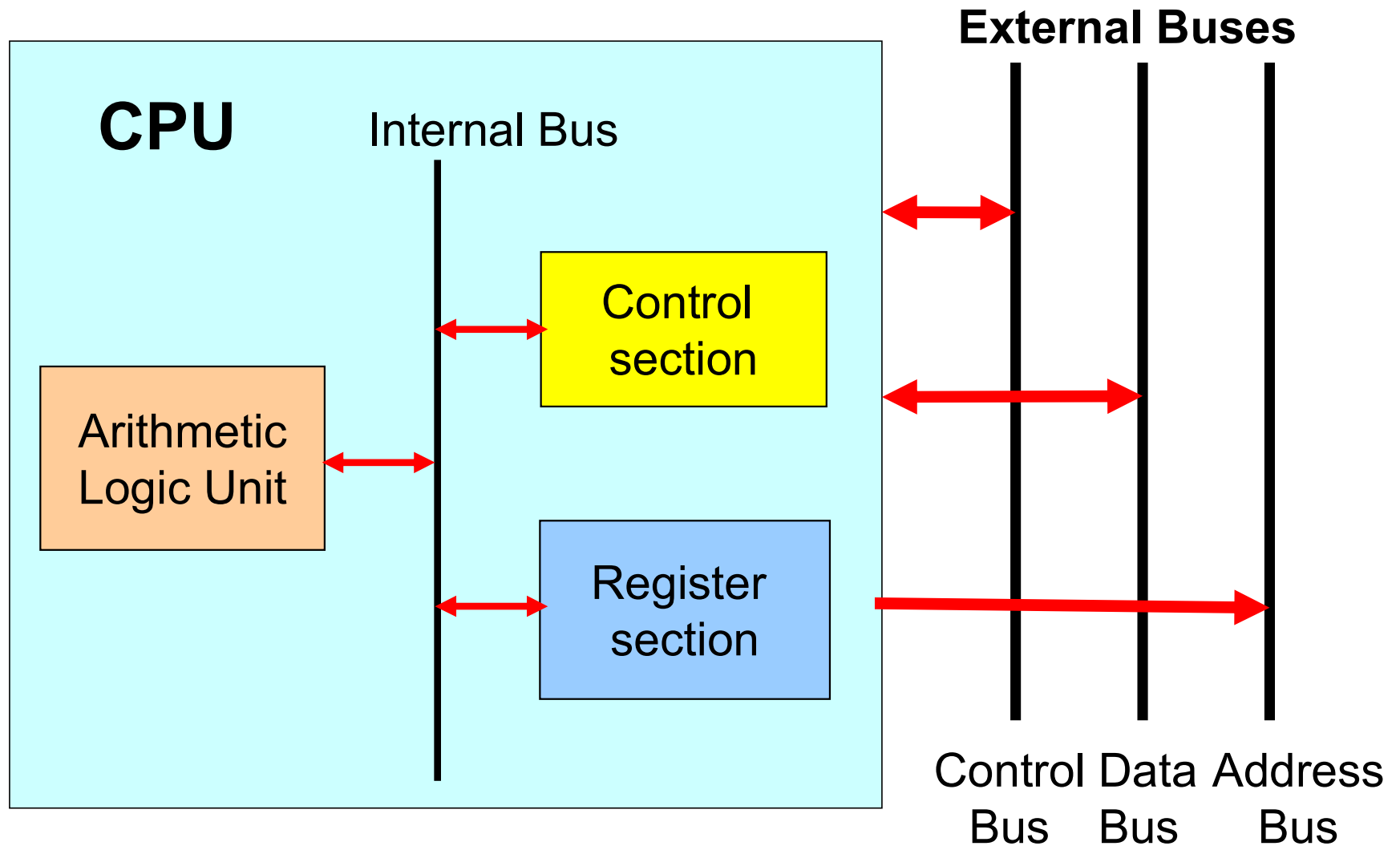
1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

The Role of the Processor

- What is the role of the processor (CPU)?
 - **Fetch instructions** - CPU read instructions from memory.
 - **Decode instructions** - Instruction is decoded in order to determine action required.
 - **Fetch data** – Some data from memory may need to be fetched in order to complete execution of instruction.
 - **Execute** – Execution of the instruction may require CPU to perform some arithmetic or logical operation on the data.
 - **Write data** – Result of execution may need to be stored in CPU temporarily or to stored back to memory.
 - This fetch-decode-execute cycle is performed repeatedly.

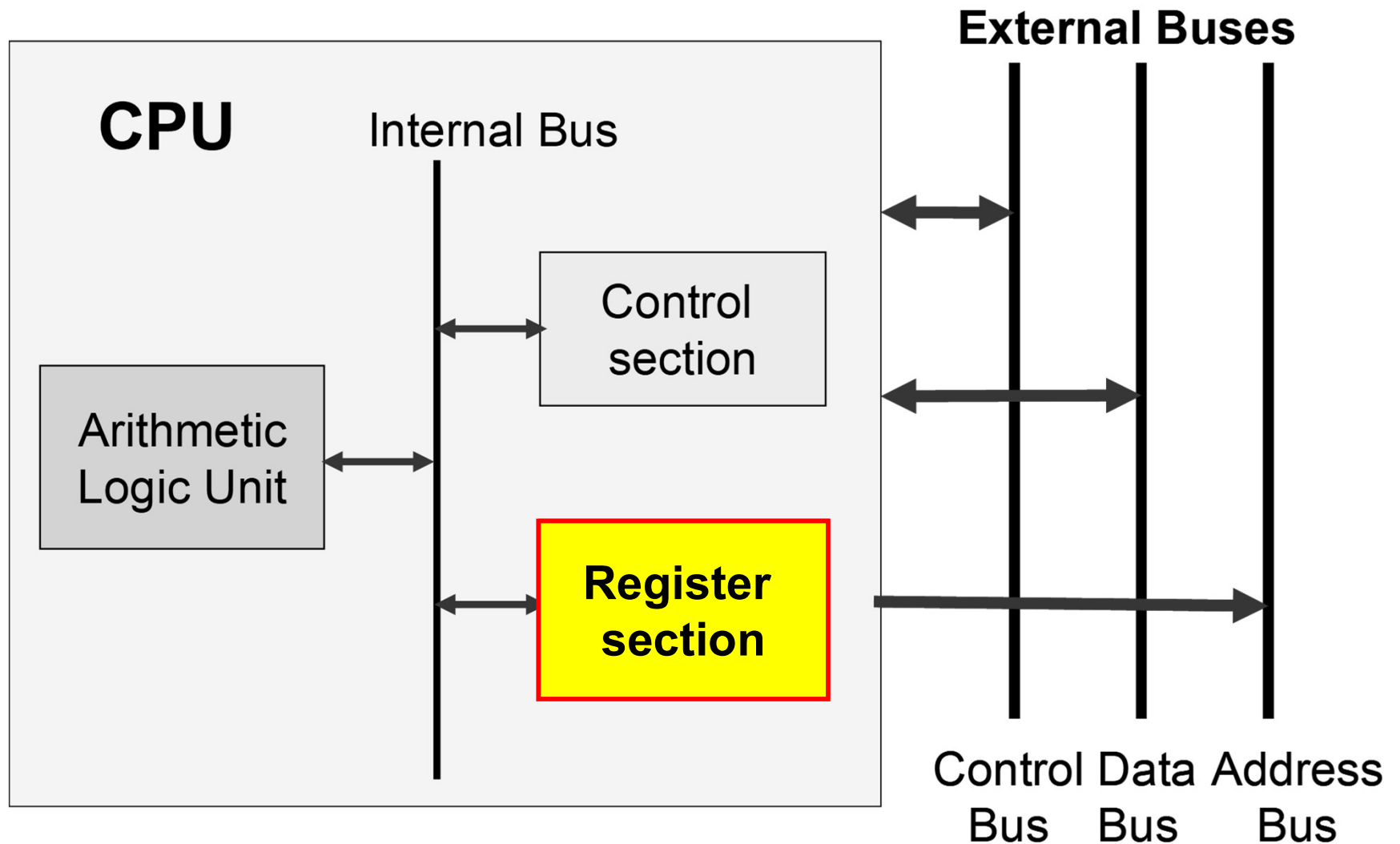
Basic Processor Organization

- The CPU can be divided into 3 functional sections:



Basic Processor Organization

- The CPU can be divided into 3 functional sections:



Processor – Register Section

User Visible Registers

Just an overview on registers!!!
More of these in the next few chapters!!!

- General user visible registers
- **Data register** – used to hold data temporarily during CPU operations.
- **Address register** – used to hold addresses of operands in memory.
- **Stack pointer** – a special register that is used to manage a stack in memory.

Processor – Register Section

Special Registers

- Special user visible registers
 - **Status register** – contains the current status of the CPU and a set of 1-bit flags that indicates the outcome of the ALU operations. This set of flags is known as the condition code register (**CCR**)
 - **Program counter** – contains the address of the next instruction to be executed.
- The PC automatically increments after the execution of the current instruction.

Processor – Register Section

Condition Code Flags

Common condition code flags

- **Negative (N)** – this flag is set whenever the most significant bit (MSB) of the result is 1.
- **Zero (Z)** – this flag is set whenever the result is zero.
- **Overflow (V)** – this flag is set whenever the result cannot be represented by the two's complement range of number representation.
- **Carry (C)** – this flag is set when the result of an addition causes a carry at the MSB or a subtraction causes a borrow at the MSB.

Processor – Register Section

Condition Code Flags

Overflow (V) Flag in Arithmetic Operations

Addition: e.g. $A+B=C$

- set if $(A_{\text{Sign}} = B_{\text{Sign}}) \neq C_{\text{Sign}}$
- Why? Addition of two positive numbers cannot give a negative result

Subtraction: e.g. $A-B=C$

- set if $(C_{\text{Sign}} = B_{\text{Sign}}) \neq A_{\text{Sign}}$
- Why? Subtraction of a negative number from a positive number cannot give a negative result

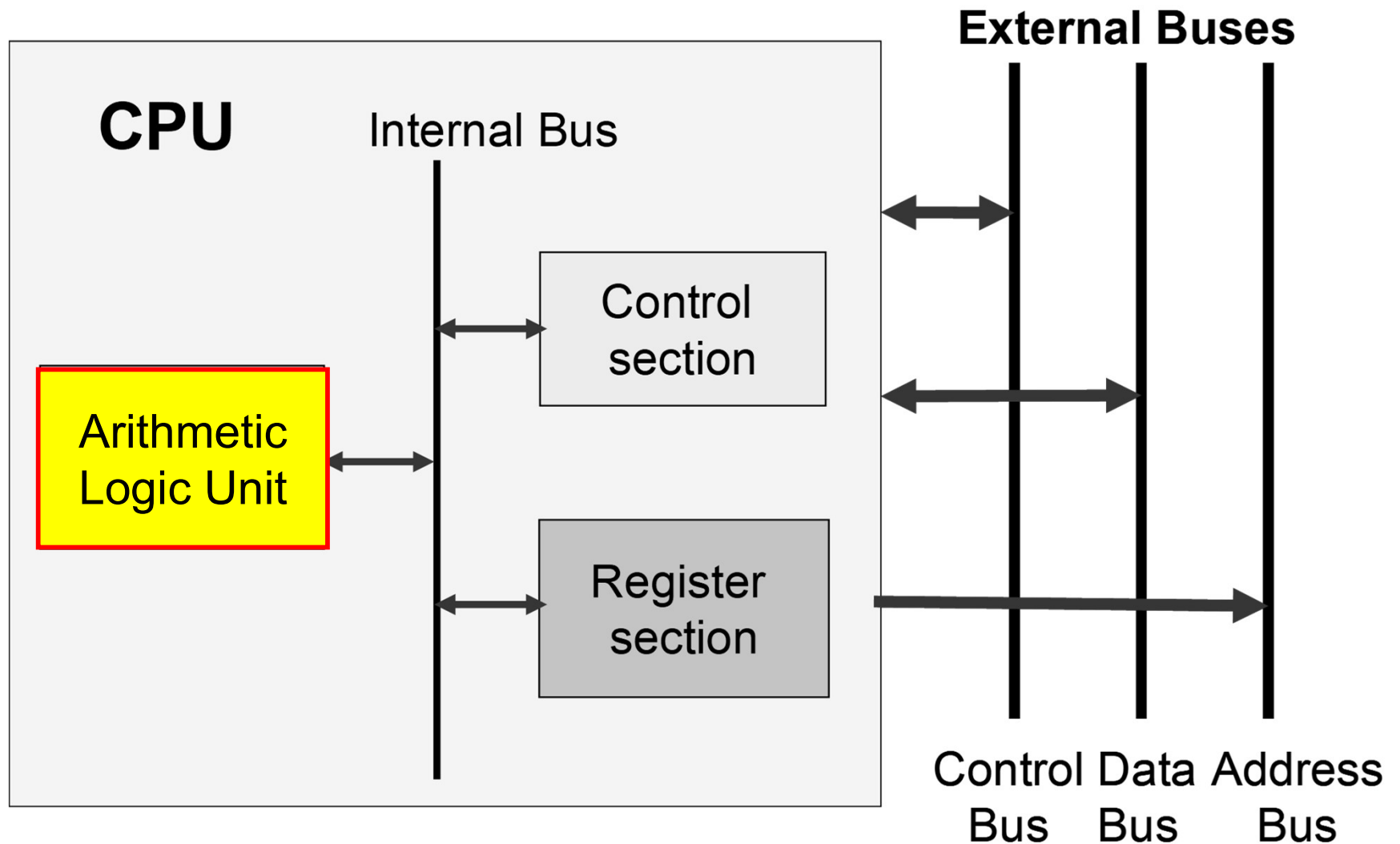
Processor – Register Section

Non-user Visible Registers

- Control registers (non-user visible, used by the CPU internally)
- **Instruction register** – holds the opcode of the current instruction that is being executed.
- **Temporary or buffer registers** – holds the address or data internally during intermediate stages of CPU operations or instruction execution (e.g. arithmetic operations, I/O, interrupts, etc.)

Basic Processor Organization

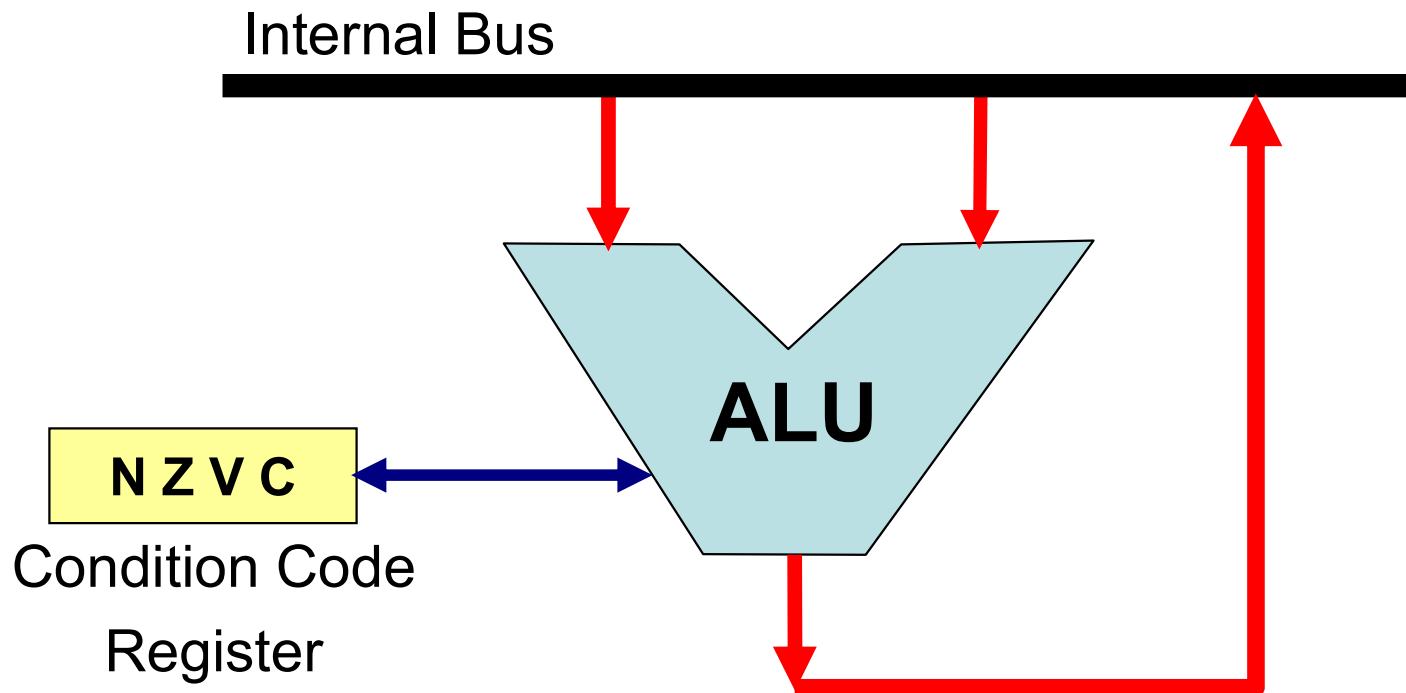
- The CPU can be divided into 3 functional sections:



Processor – ALU

Arithmetic Logic Unit

- This section performs the arithmetic and logical operations specified by an instruction.

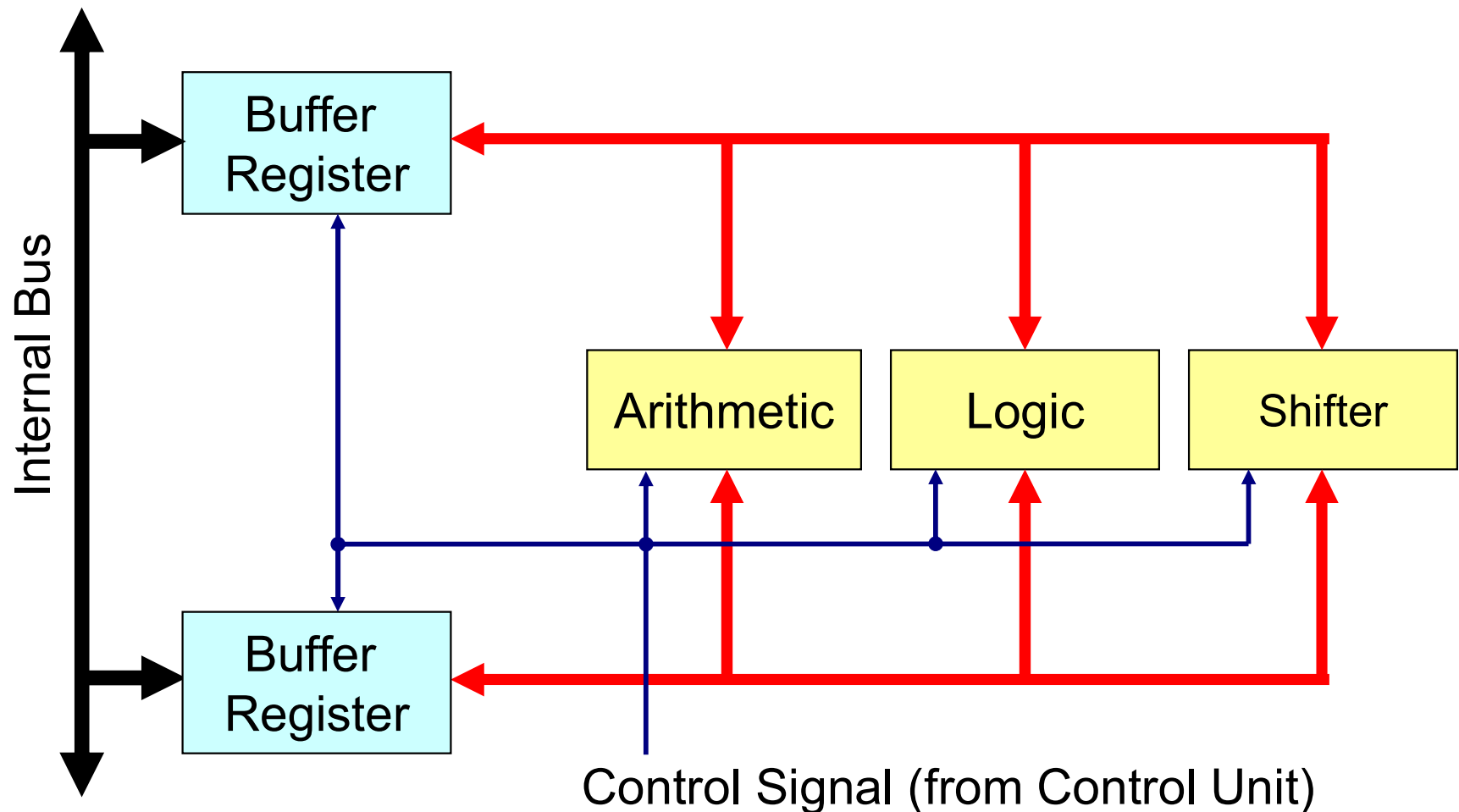


A 32-bit CPU has 32-bit internal bus

Processor – ALU

Typical ALU Structure

- An ALU contains buffer registers for temporary storage of input operands and result.

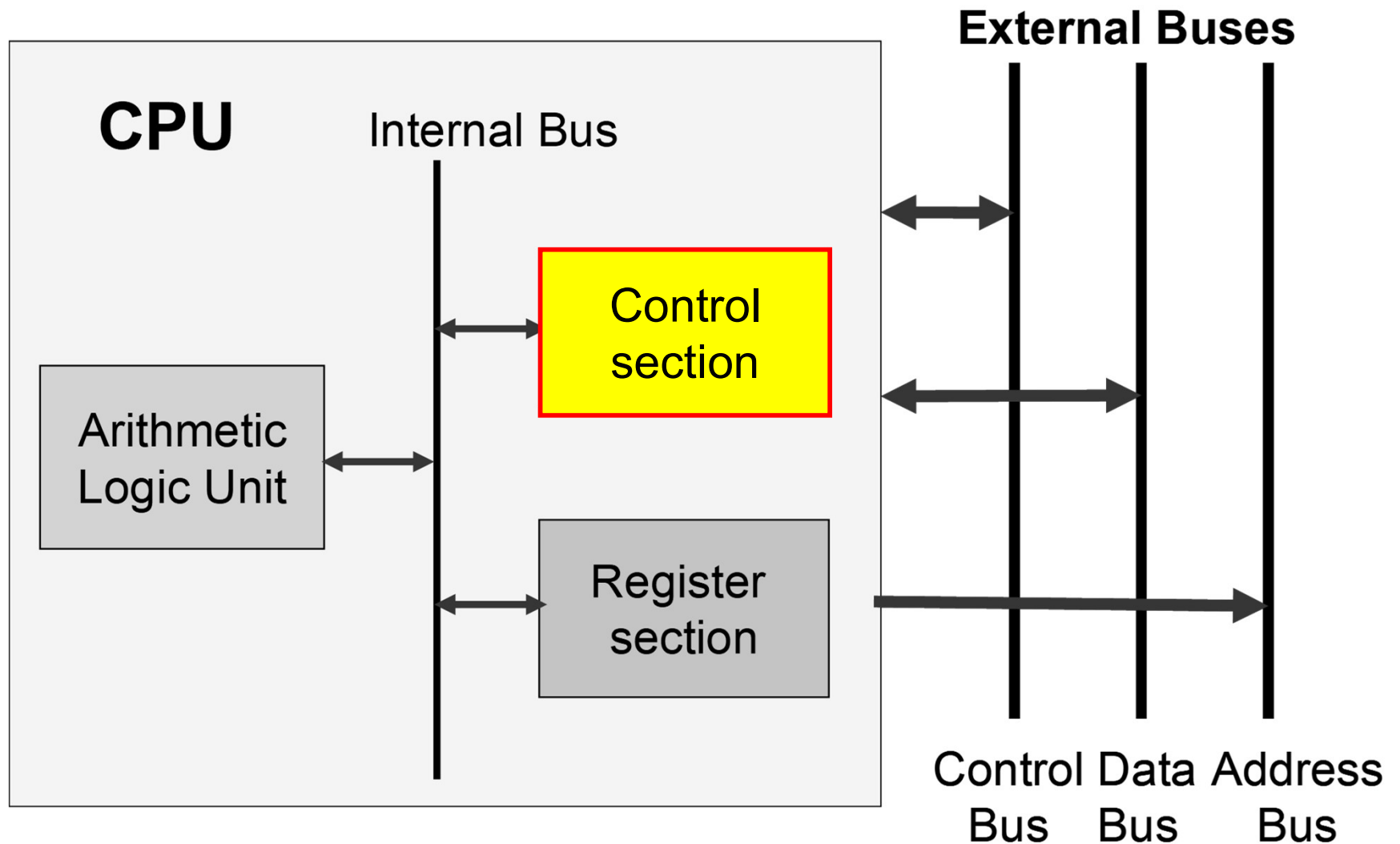


ALU's Functional Elements

- The **ALU** also contains functional circuits that are able to perform the different requirements of the microprocessor's instructions set.
- **Arithmetic Unit** – performs math-type operations (add, subtract, divide and multiply).
- **Logic Unit** – performs logical operations (AND, OR, XOR).
- **Shift Unit** – performs bit shift and rotation.
- Many of this operations can influence the **N, Z, V** and **C** flags in the **CCR**.

Basic Processor Organization

- The CPU can be divided into 3 functional sections:



Processor – Control Section

Control Unit

Some of the roles of the control unit are:

- **Decoding instructions** – it decodes opcode fetched into the instruction register into internal and external control signals needed for execution.
- **ALU** – it activates specific ALU functions based on the decoded signals from the instruction register.
- **Movement of data** – it controls movement of data between memory-registers and or internally between registers .
- **External signals** – it handles external signals into the CPU such as interrupts and reset.

Processor – Control Section

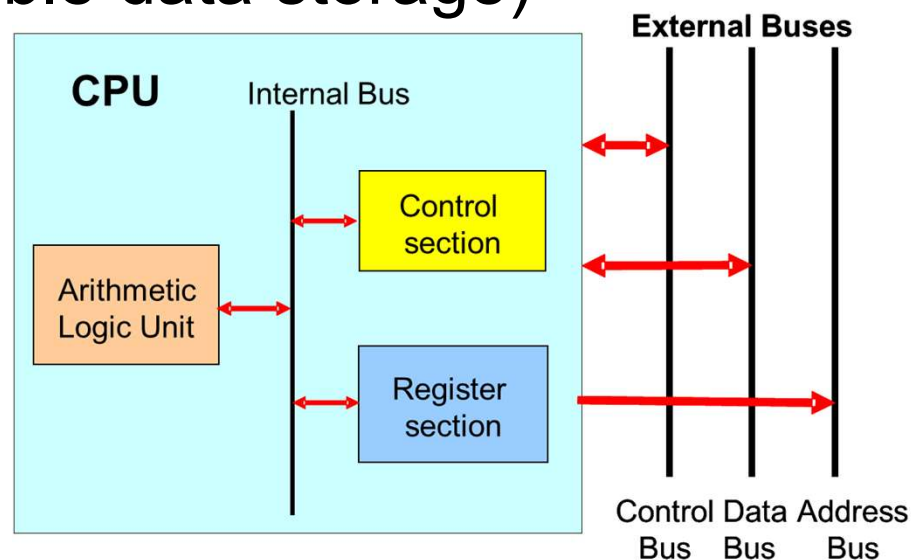
Control and Sequencing

- Operation of the CPU is synchronized by a master clock
- CPU can be viewed as a sequential state machine.
- Each instruction has a number of micro-operations associated with it. Each micro-operation is performed at state change of the CPU clock.
- The frequency of this CPU clock can range from MHz to a few GHz.

Processor

Impacts of bus widths

- Impact on the various bus width
- **CPU Internal Bus:** A 32-bit CPU have a 32-bit internal bus (i.e. impacts the number of bits a CPU can process in one cycle)
- **Data Bus:** How much data can be transferred in one cycle (i.e. speed of data transfer)
- **Address Bus:** How large is the address space (i.e. the size of the possible data storage)



Chpt 2: The Central Processing Unit (CPU)

Chapter Contents



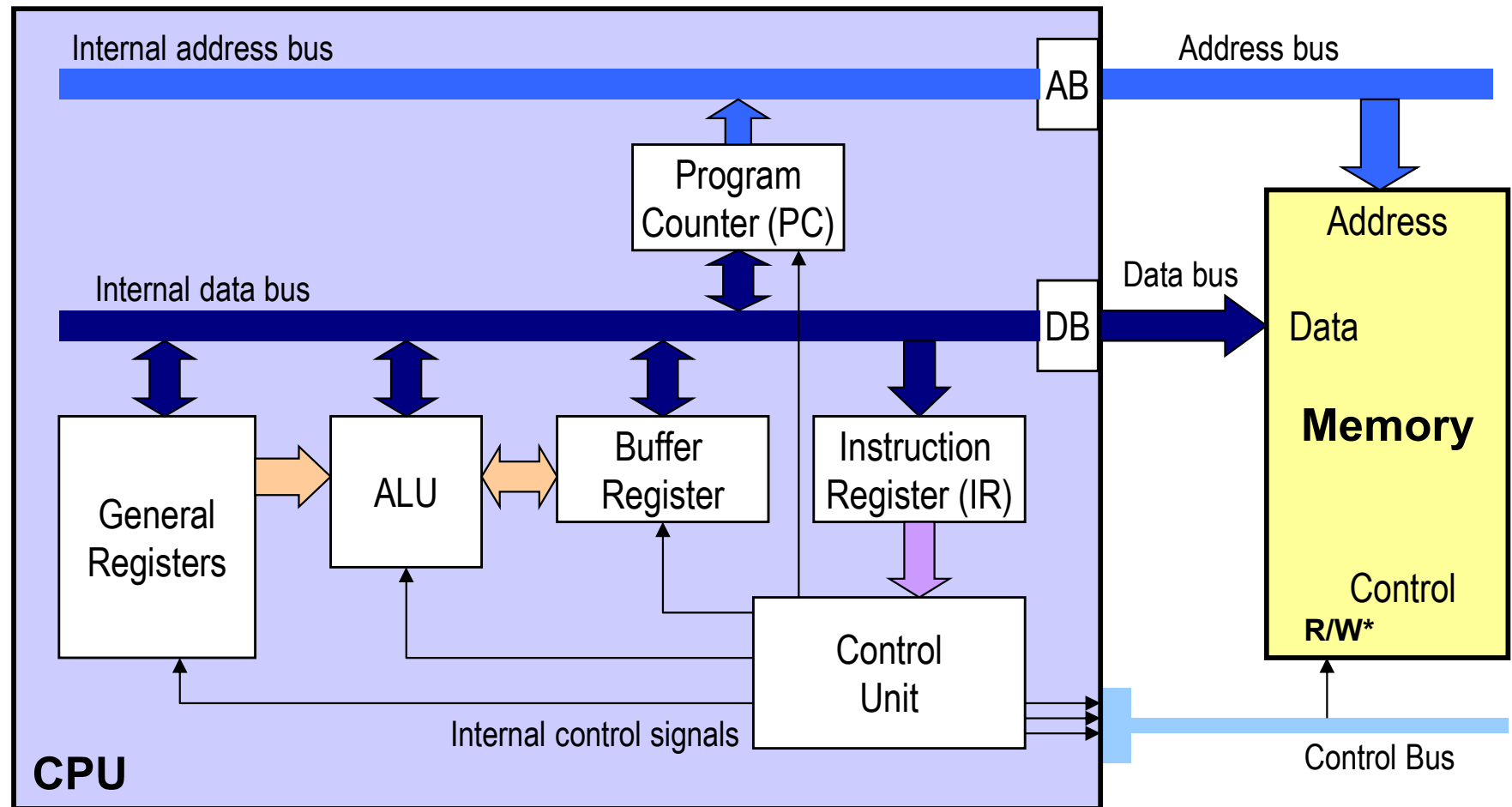
<https://ict1011.stevenwong.org/c2t7/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Basic Execution Cycle

A Simple Processor

Basic components of a simple processor (CPU) and its interface signals to external main memory.



AB = Address buffer/latch

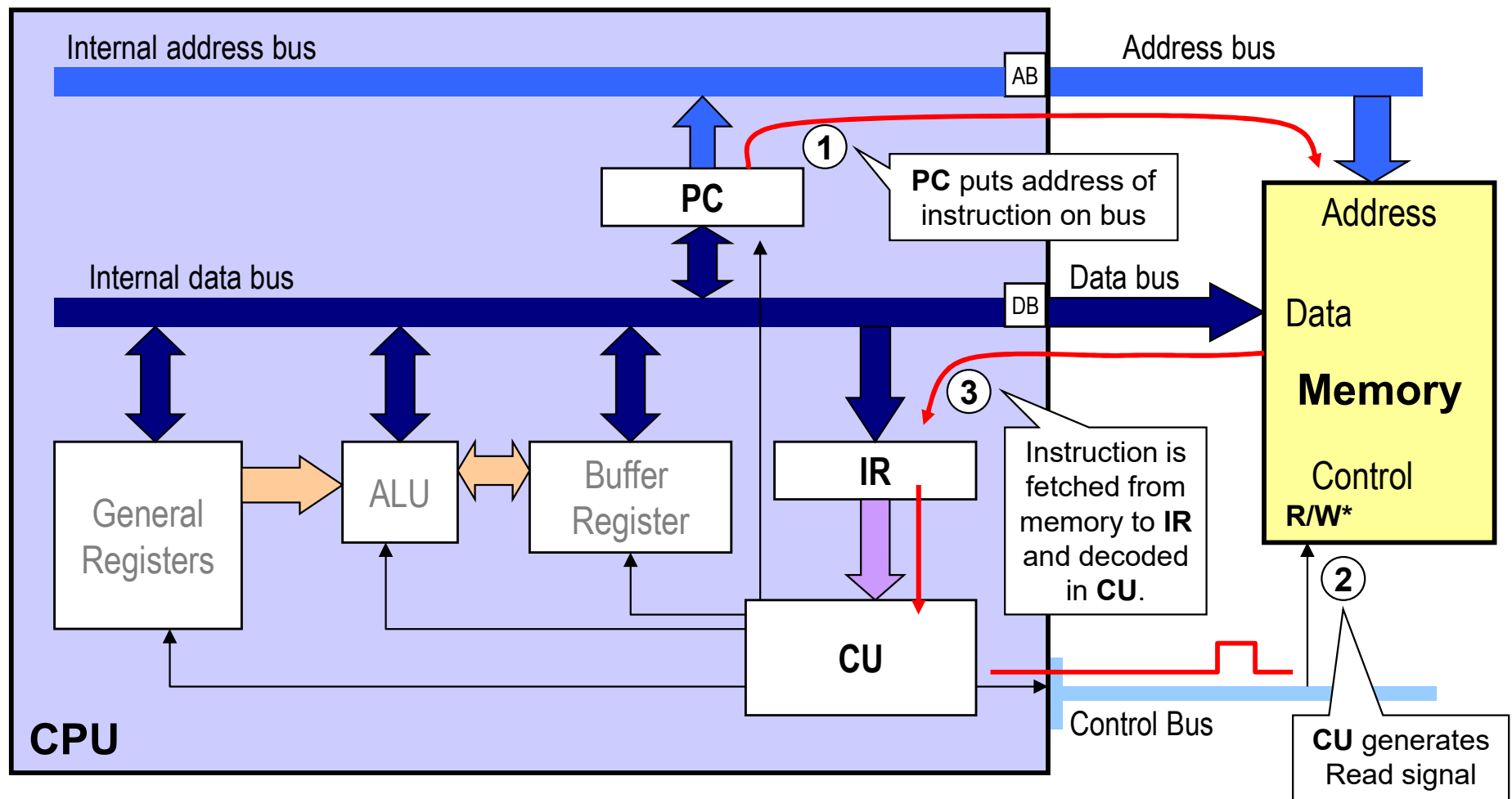
DB = Data buffer

Chpt 2: The Central Processing Unit (CPU)

Basic Execution Cycle

Fetch Cycle - Instruction

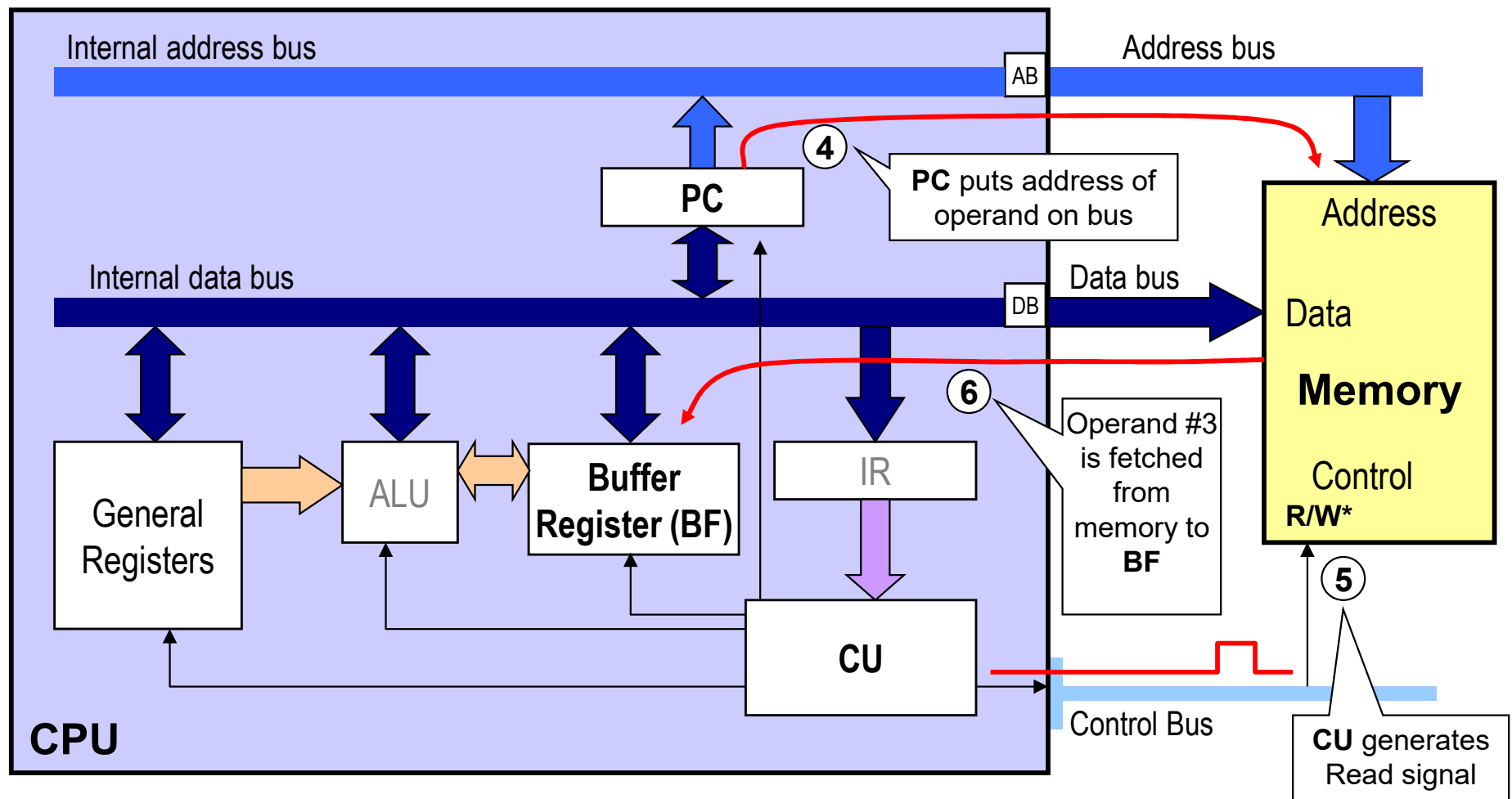
- Consider an instruction like: add.w #3, R1
In fetch cycle, the **PC** points to address of next the instruction and its opcode is fetched into the **IR**.



Basic Execution Cycle

Fetch Cycle - Operand

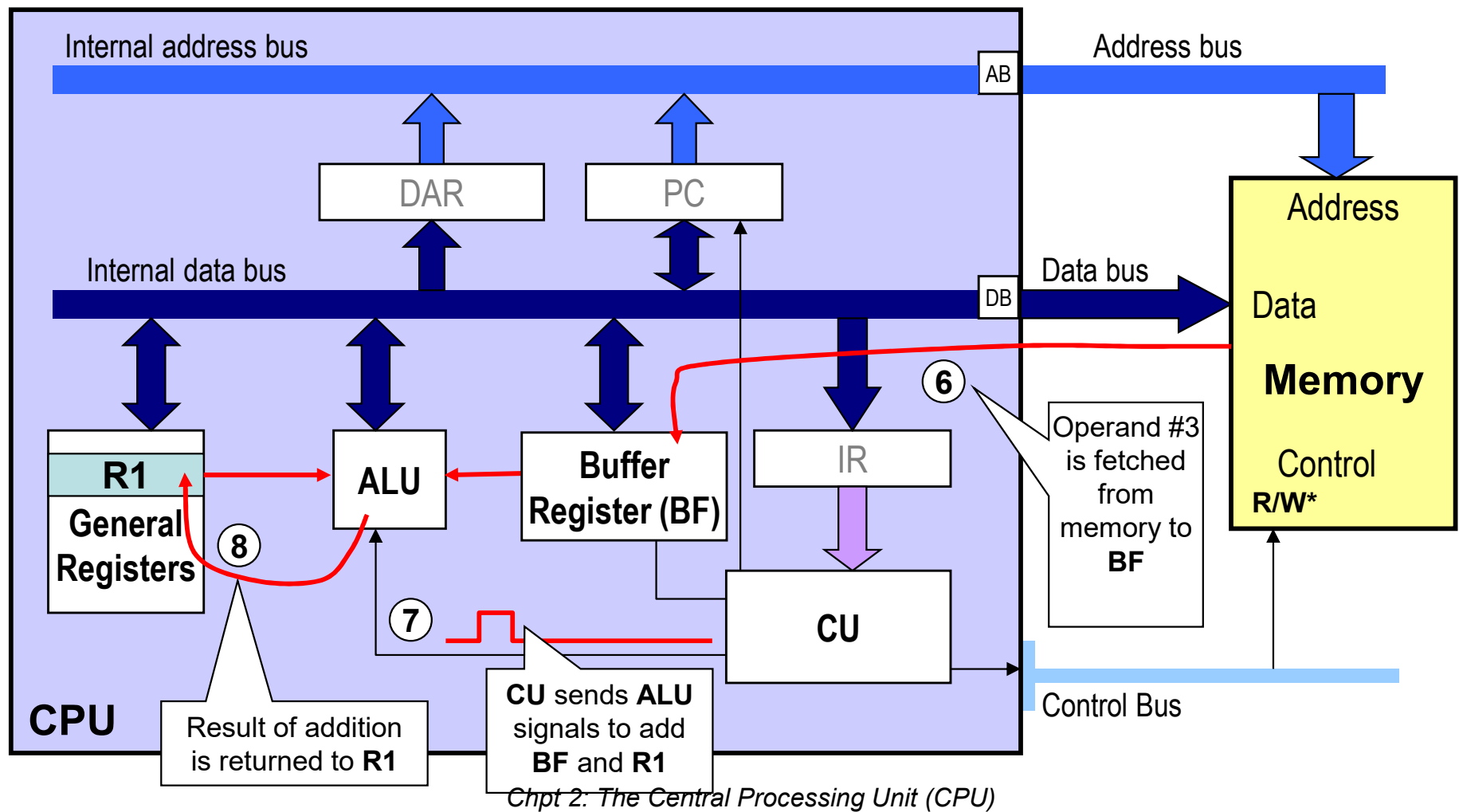
- Decoding `add.w #3, R1` suggest an operand is needed. Since its is stored in memory, another fetch is required to retrieve operand into CPU.



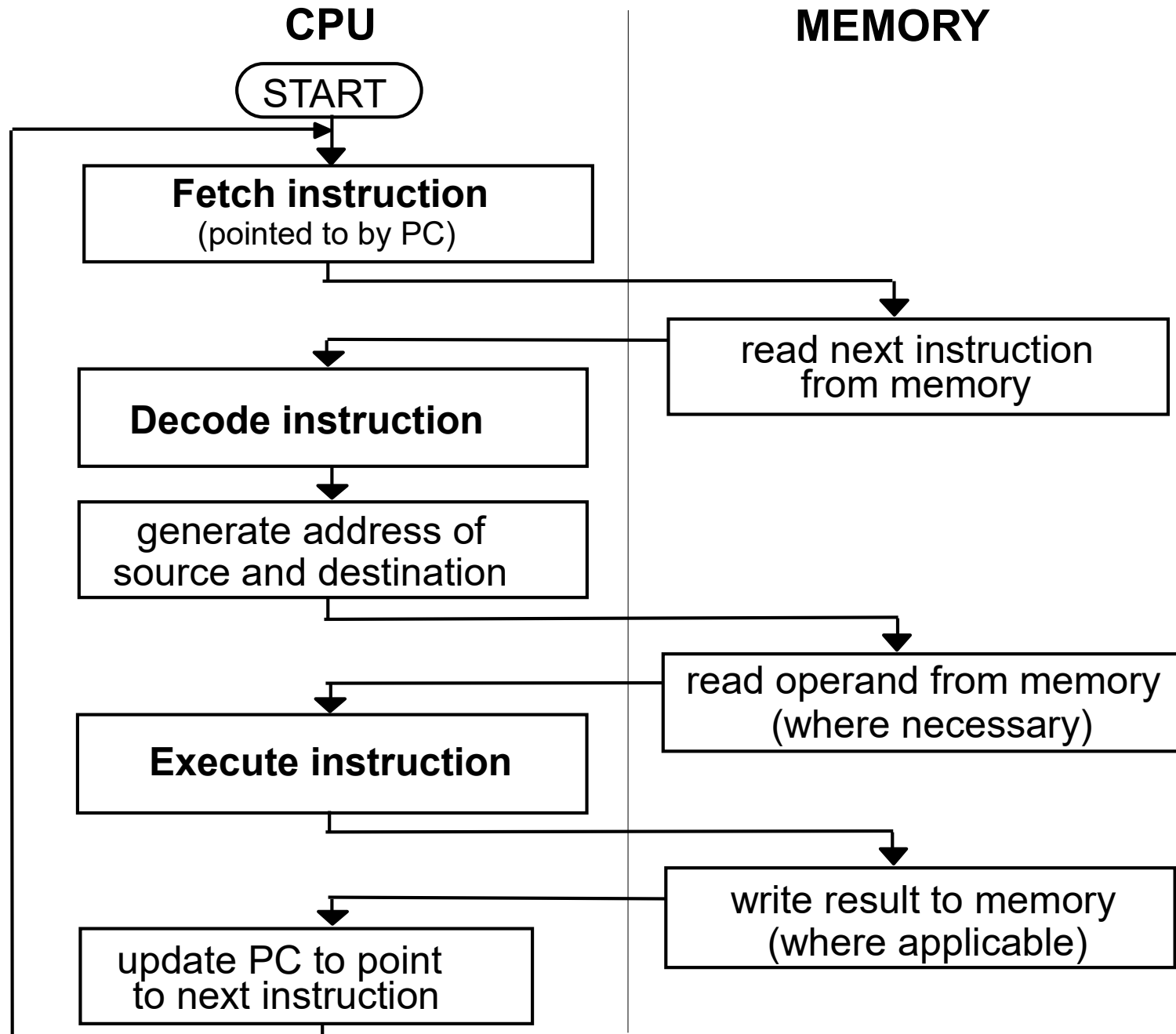
Basic Execution Cycle

Execute Cycle - Add

- In the execution cycle, the operand #3 and the contents in register **R1** are added and the result is stored back into **R1**, replacing its original content.



Typical Instruction Execution Flowchart



Observations of Program Execution

- Execution of a single instruction may involve multiple accesses to memory. In most single memory (von Neumann-type) CPU, different instructions take **different number** of clock **cycles** to execute.
- Data transfer on external bus is slower than within CPU's internal bus. μ P system performance is limited by data traffic bandwidth between CPU and memory (also known as the **von Neumann bottleneck**).
- Keeping regularly used operands in CPU registers helps reduce memory access.
- Keeping instructions and data in separate memories (**Harvard architecture**) can help make instructions execute in more regular cycles and thus improve performance.

Chapter Contents

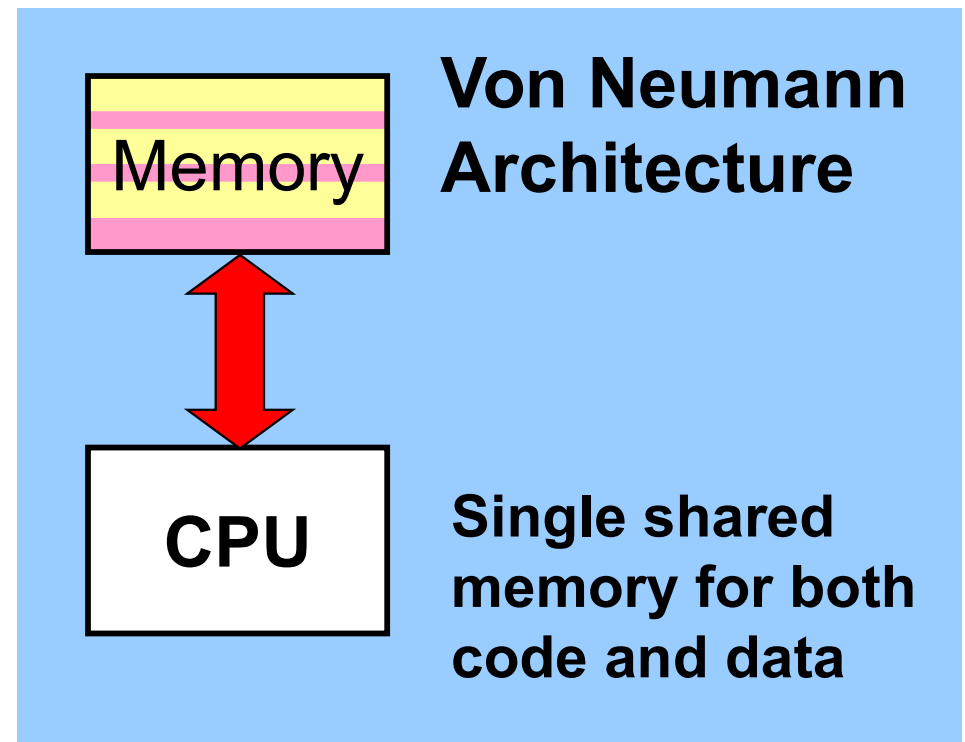
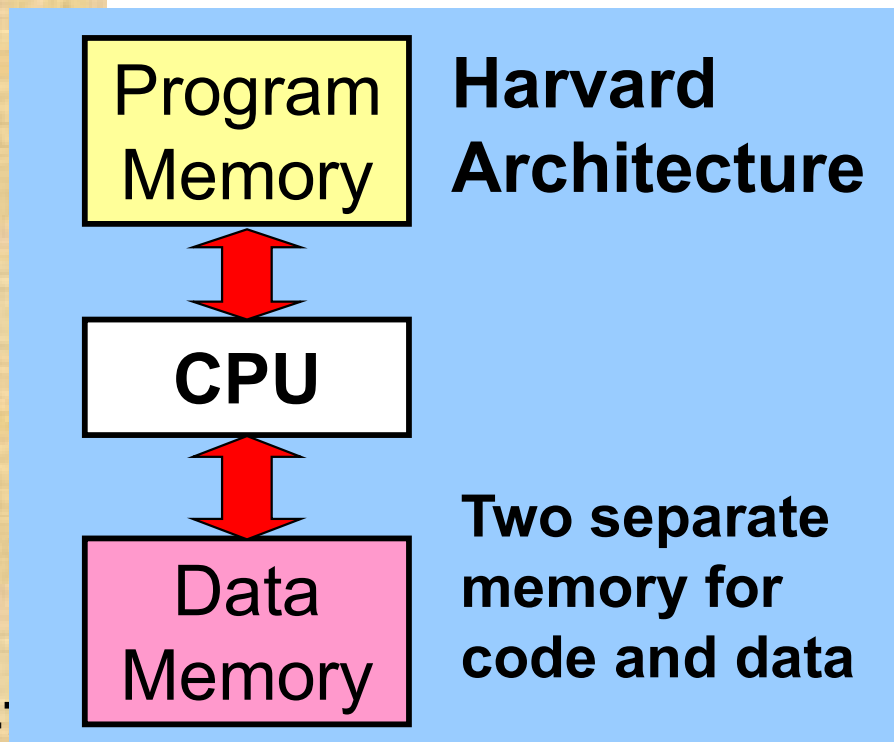


<https://ict1011.stevenwong.org/c2t8/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Harvard versus Von Neumann

- Two major classes of computer architecture:
- Harvard architecture** - named after Harvard series of relay calculators developed by Aiken at Harvard U.
- Von Neumann architecture** - developed by John Von Neumann at **Princeton** U. Influenced ENIAC's design.

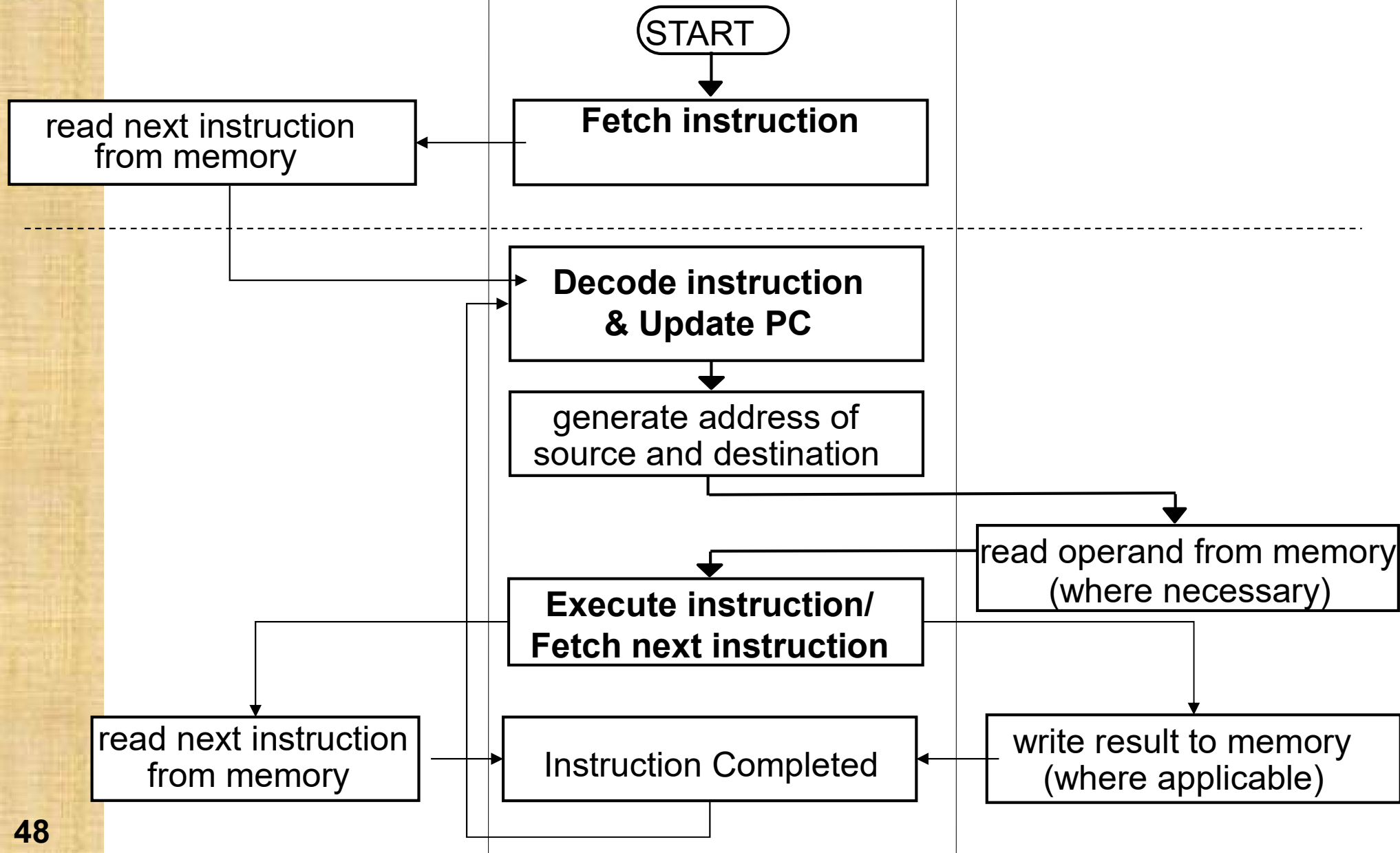


Harvard Architecture (An example)

PROGRAM MEMORY

CPU

DATA MEMORY



Chapter Contents



<https://ict1011.stevenwong.org/c2t9/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

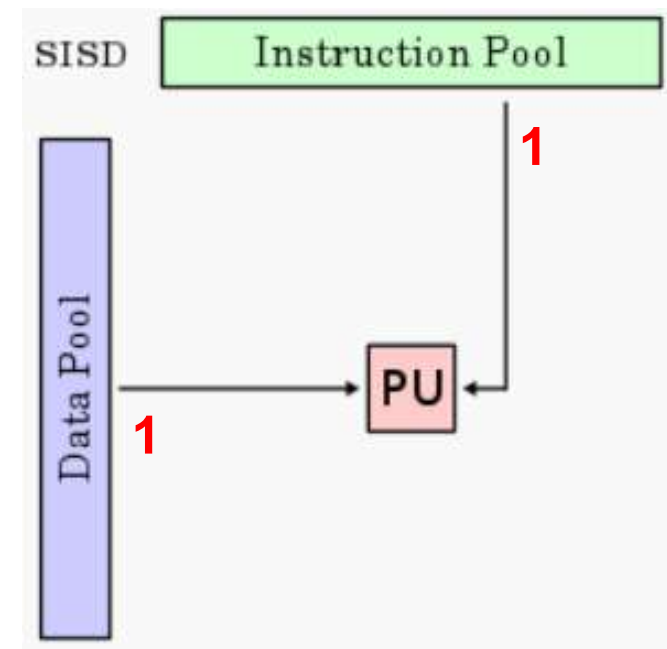
Different Computer Architectures

Flynn's Taxonomy

Four classifications based upon the number of concurrent instruction (or control) and data streams available in the architecture

SISD: Single Instruction, Single Data

A single instruction operating on a single SET of data



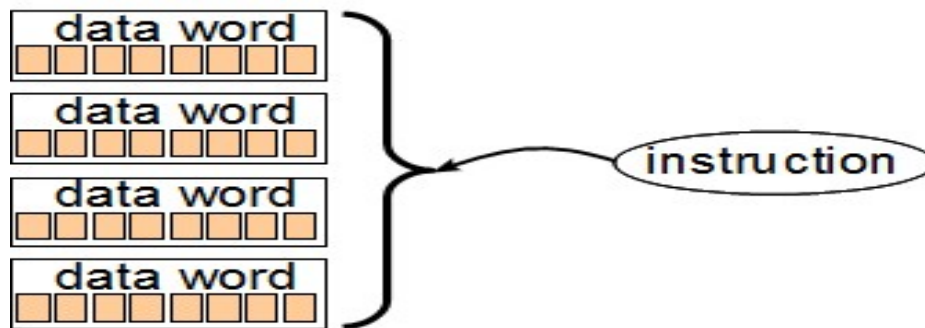
E.g. Simple uni-core computers

Different Computer Architectures

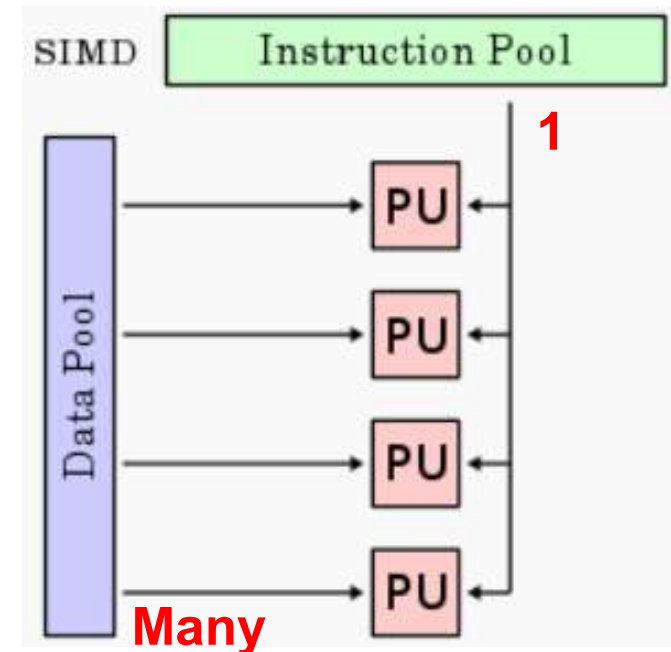
Flynn's Taxonomy

SIMD: Single Instruction, Multiple Data

A single instruction operating on multiple SETs of data



E.g. Graphics processing

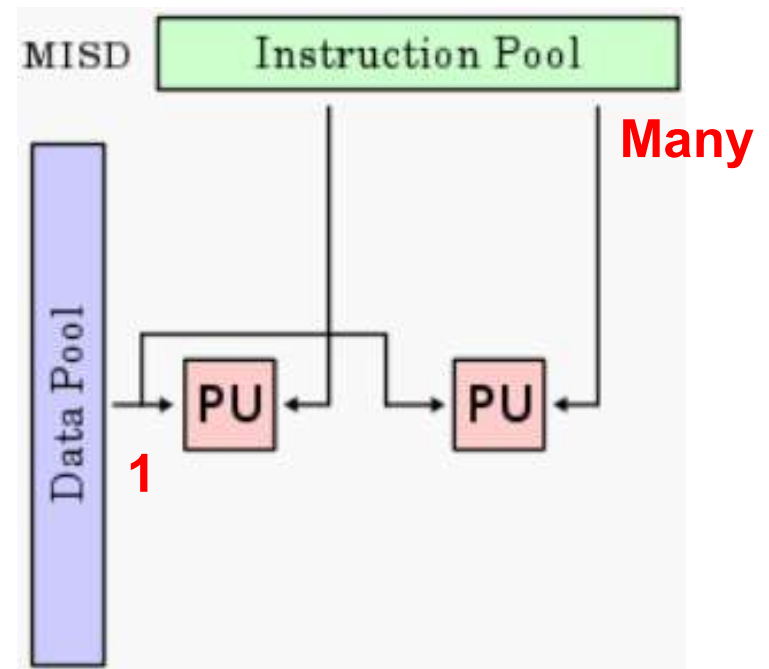
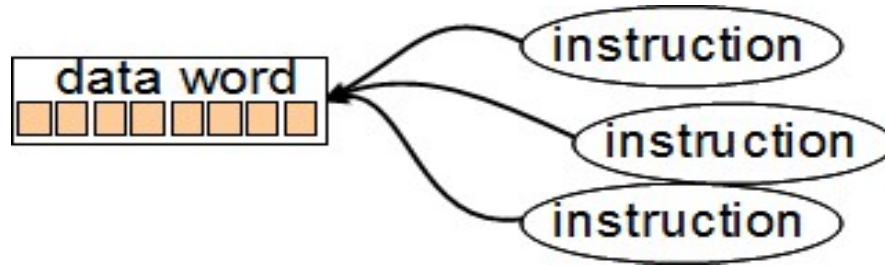


Different Computer Architectures

Flynn's Taxonomy

MISD: Multiple Instruction, Single Data

Multiple Instructions operating on a single SET of data simultaneously



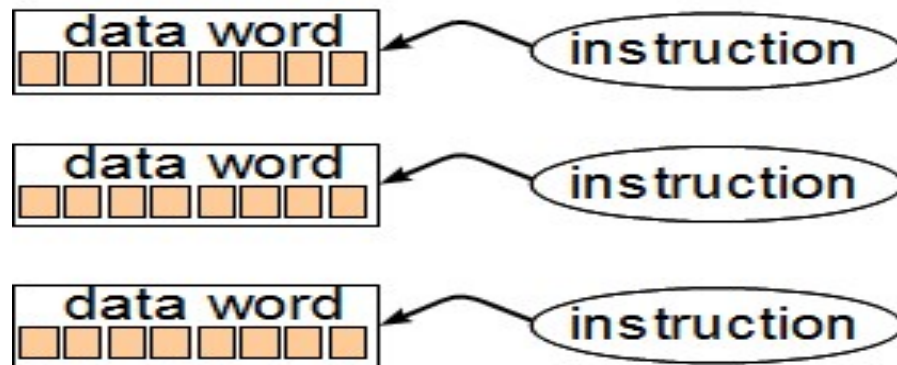
E.g. Fault Tolerant Systems

Different Computer Architectures

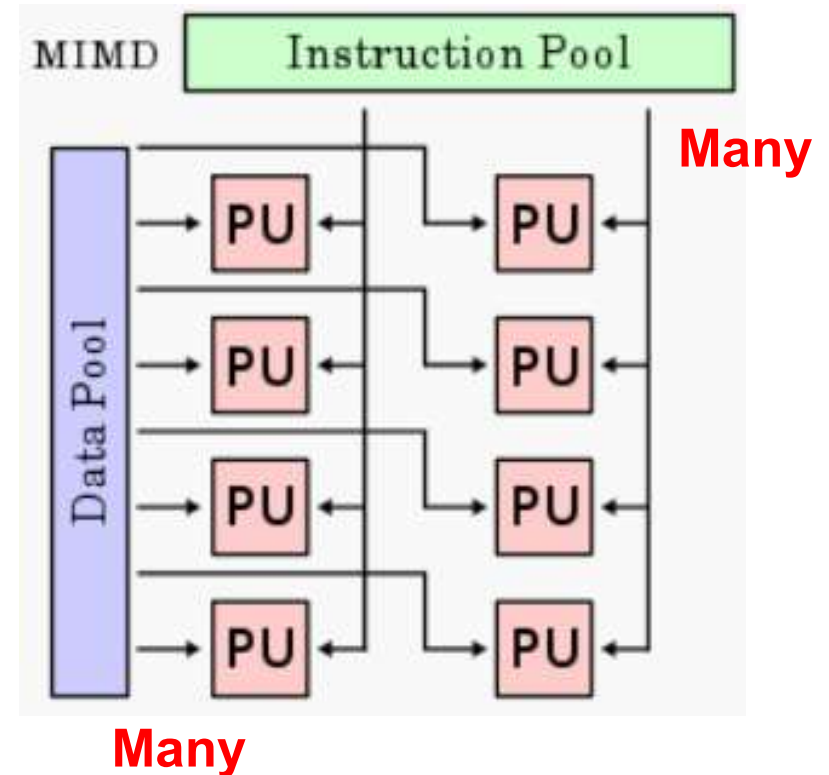
Flynn's Taxonomy

MIMD: Multiple Instruction, Multiple Data

Multiple Instructions operating on multiple SETs of data simultaneously



E.g. Multi-core Processors



Chapter Contents



<https://ict1011.stevenwong.org/c2t10/>

1. **Approaches to Computing**
 - Programming by hardware versus by software
 - von Neumann Architecture
 - The Instruction Set Architecture (ISA)
2. **Basic components of a microcomputer**
 - Memory
 - Input/Output Interfaces
 - CPU
4. **Program execution**
 - A typical instruction execution cycle
5. **Different Computer Architectures**
 - Harvard (vs von Neumann)
 - Flynn's Taxonomy
 - RISC and CISC

Different Computer Architectures

CISC and RISC

CISC: Complex Instruction Set Computer

Most Intel processor up to the Pentium are considered CISC. One instruction requires the CPU to perform multiple complex operations.

Typical characteristics of a CISC

- Complex instruction-decoding logic, driven by the need for a single instruction to support multiple addressing modes.
- Variable length instructions where the length often varies according to the addressing mode
- Instructions which require multiple clock cycles to execute.

Different Computer Architectures

CISC and RISC

RISC: Reduced Instruction Set Computer

Most modern non-Intel processor, such as the ARM and MSP430, are RISC. One instruction usually performs a smaller number of operations.

Typical characteristics of a RISC

- Reduced instruction set.
- Less complex, simple instructions.
- Hardwired control unit and machine instructions.
- Few addressing schemes for memory operands with only two basic instructions, LOAD and STORE

Different Computer Architectures

CISC vs RISC

CISC

- Emphasis on hardware
- Includes multi-clock complex instructions
- Allows Memory-to-Memory transfers: “LOAD” and “STORE” incorporated in instructions
- Small code size, high cycles per instruction
- Transistors used for storing complex instructions

RISC

- Emphasis on software
- Single-clock, reduced instructions only
- Memory “LOAD” and “STORE” are usually independent instructions
- Larger code size, low cycles per instruction
- Spends more transistors on memory registers

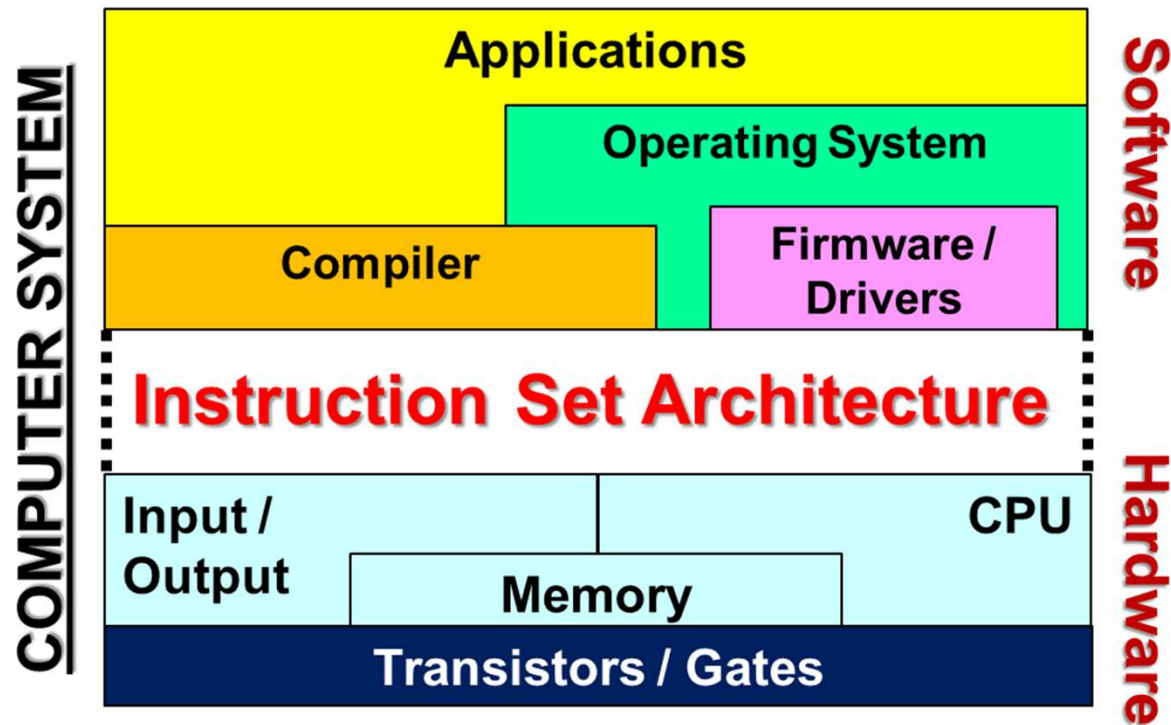
The line is blurring. New processor, such as the new Intel processors, uses a Hybrid model with CISC-like instructions running on a RISC-like processor.

Chapter Contents



<https://ict1011.stevenwong.org/c2t11/>

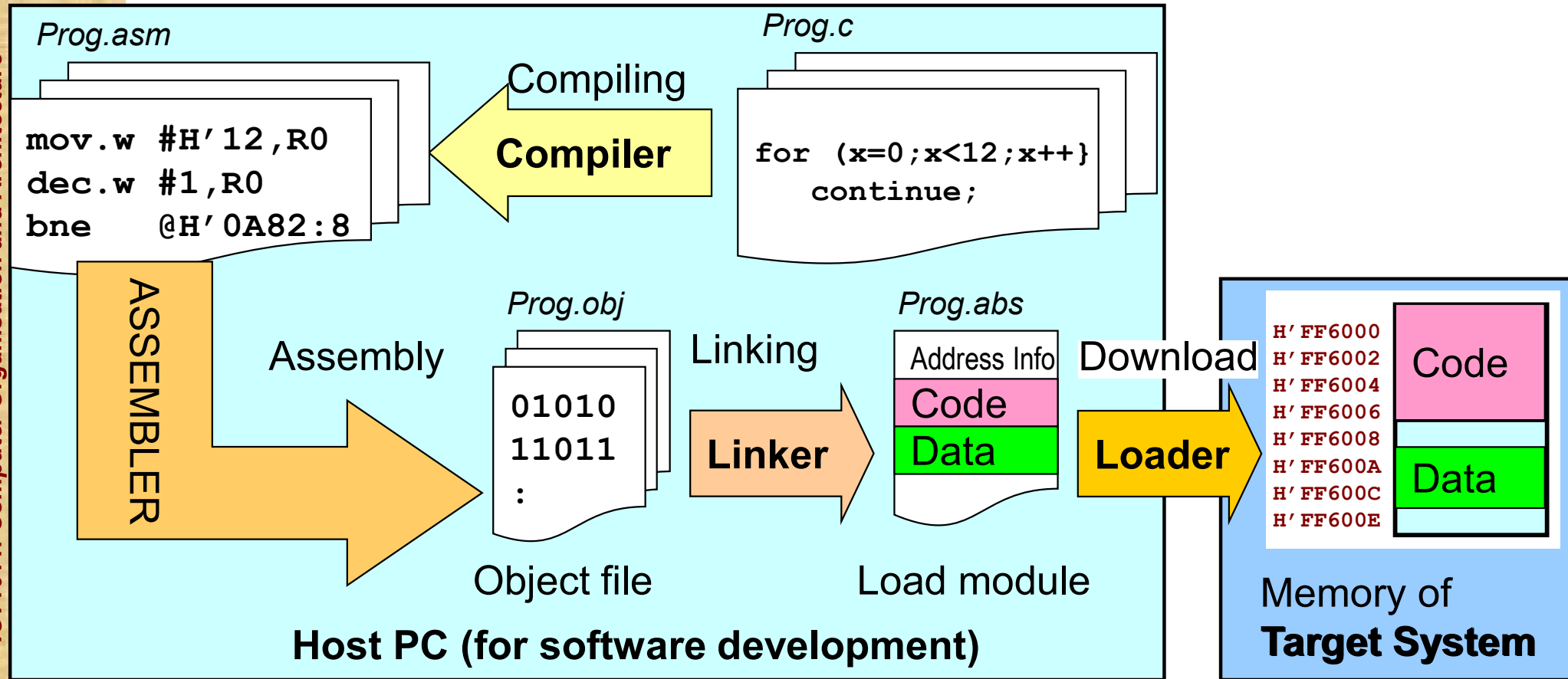
6. Program Development Process



Program Development Process

An overview

Roles of Compiler, Assembler, Linker and Loader



NEXT

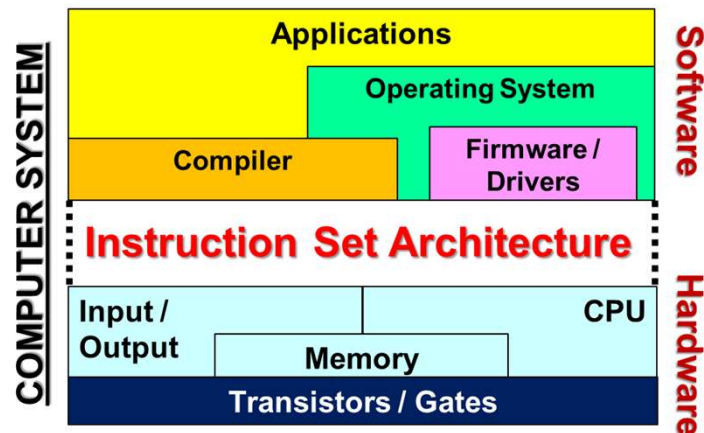
Summary



<https://ict1011.stevenwong.org/c2s/>

LEARNING OUTCOMES OF CHAPTER:

- ✓ Knowledge about hardware implementation of a computer
- ✓ Understand how instructions are actually executed on a computer
- ✓ Ability to distinguished between the various different computer architectures



**NEXT UP:
Coding in
Assembly**